HotelFlow

Szabó Máté 13.D
Monostori Márk György 13.D
2025.10.03.

Tartalomjegyzék

1. Bevezetés 9
1.1 A feladat rövid ismertetése 9
2. Elvárások 10
2.1 Platform és működés 10
2.2 Használt technológiák 10
2.3 Mit kell átadni 10
2.4 Fejlesztési szabályok 10
2.5 Modulok 10
2.6 Fejlesztési lehetőségek 11
3. A rendszer működése 11
3.1 Vendégélmény 11
3.2 Szállodai adminisztráció 11
3.3 Központi adminisztráció 11
3.4 Foglalás és fizetés folyamata 12
4. Nem funkcionális követelmények 12
5. Dokumentáció 12
6. Összegzés 12
7. Projekt adatlap 12
8. Bevezetés 14
9. Táblák 14
9.1 users 14
9. 2 password_reset_tokens 15
9.3 sessions 15
9.4 two_factor_recovery_tokens 15
9.5 hotels 16
9.6 rooms 16
9.7 images 17
9.8 imagesRelation 17
9.9 services 17

9.10 serviceTags 18
9.11 hotelTagRelation 18
9.12 roomTagRelation 18
9.13 bookings 18
9.14 bookingsRelation 19
9.15 guests 19
9.16 invoices 20
9.17 booking_payments 20
9.18 booking_invoice_details 21
9.19 servicesRelation 21
9.20 rfidKeys 21
9.21 rfidKeyConnection 22
9.22 rfid_assignments 22
9.23 devices 22
9.24 pending_handshakes 23
9.25 Adatbázis EK diagram 24
10. Bevezetés 26
11. Bejelentkezés (Login) 26
12. Regisztráció (Register) 27
13. Szállásfoglalás (Booking) 27
14. Szállodai oldal – foglalás visszaigazolása 28
15. Kártyás beléptető rendszer (QR kód és RFID) 29
16. Szoba felvétele a szállodához 29
17. Szálloda adatbővítése / szerkesztése 30
18. Szoba elérhetőségének ellenőrzése 30
19. Foglalás árkalkulációja 31
20. A tesztelés célja 34
21. Tesztkörnyezet 34
21.1. Hardver környezet 34
21.2. Szoftver környezet 34

21.3. Böngészők 34
21.4. Backend futtatási környezet 35
21.5. Frontend környezet 35
22. Tesztelési módszerek 35
22.1. Manuális tesztelés 35
22.2 Automatizált tesztelés (Selenium) 35
22.3. API tesztelés 35
23. Tesztesetek 36
23.1. Frontend tesztesetek 36
23.2. API tesztesetek 39
24. Hibakezelés értékelése 44
24.1. Validációs hibák kezelése 44
24.2. API hibakezelés 44
25.3. Frontend hibakezelés 45
25. Alapvető biztonsági tesztelés 45
25.1. Szerepkör-alapú hozzáférés-vezérlés 45
25.2. Token-alapú hitelesítés 45
25.3. Jogosulatlan hozzáférési kísérletek kezelése 46
25.4. Adatvédelem 46
26. További fejlesztési javaslatok 46
27. Következtetés 46
28. Rendszer architektúra 48
28.1 Általános felépítés 48
28.2 Fő komponensek 48
28.2.1 Kliens–szerver működés (folyamat-ábra szövegesen) 48
28.3 Frontend–backend kapcsolat 48
29. Felhasznált technológiák 48
29.1 Backend technológiák 48
29.2 Frontend technológiák 49
29.3 Verziók / függőségek (a kódból) 49

30. Backend dokumentáció 52
30.1 Backend projekt struktúra 52
30.2 Adatbázis felépítése 52
4.2.1 Táblák összefoglalója (teljes lista) 52
30.3 Modellek / entitások 54
30.4 API végpontok 54
4.4.1 Auth végpontok 54
30.4.2 Foglalás (bookings) végpontok 55
30.4.3 Számla / fizetés (invoices) végpontok 56
30.4.4 Szálloda (hotels) végpontok 57
30.4.5 Szobák (rooms) végpontok 58
30.4.6 Szolgáltatások (services) végpontok 59
30.4.7 Címkék (tags) végpontok 60
30.4.8 Képek (images) végpontok 60
30.4.9 Keresés (search) és ajánlások (recommendations) 61
30.4.10 RFID kulcsok 61
30.4.11 Eszközök 62
30.4.13 Példa kérés/válasz formátumok 63
30.5 Hitelesítés és jogosultságkezelés 63
30.6.1 E-mail küldések áttekintése 64
31. Frontend dokumentáció 64
31.1 Frontend projekt struktúra 64
31.2 Komponensek 64
31.3 Routing 64
31.4 Állapotkezelés 64
31.5 API kommunikáció 64
31.6 Űrlapok és felhasználói interakciók 65
31.7 Oldalak (views) funkcionális áttekintése 65
32. Biztonsági megoldások 65
33. Telepítés (Deployment) 66

33.2 Lokális fejlesztői futtatás 66
33.3 Környezeti változók (összefoglaló) 66
34. Rendszerkövetelmények 68
35. Bejelentkezés a rendszerbe 68
35.1 Bejelentkezés lépései 68
35.2 Sikeres bejelentkezés után mi történik? 68
35.3 Hibás adatok / tipikus problémák 68
35.4 Kétfaktoros hitelesítés (2FA) 68
36. A rendszer felépítése 69
36.1 Fő felületek (szerepkörök szerint) 69
37. A rendszer fő funkciói 69
37.1 Regisztráció (Vendég vagy Szálloda admin) 69
Lépések 69
37.2 Bejelentkezés + 2FA 70
Lépések 70
37.3 Szállás keresése (Vendég) 70
Lépések 70
37.4 Csomag kiválasztása és foglalási kérelem leadása (Vendég) 70
Lépések 70
37.5 Számla és fizetés folyamata 71
37.5.1 Bankkártyás fizetés 71
37.5.2 Banki átutalás 71
37.6 Foglalásaim kezelése (Vendég) 71
Lépések – Foglalások áttekintése 71
Lépések – Vendégek hozzáadása egy foglaláshoz 71
37.7 Profil kezelése 72
Lépések – 2FA bekapcsolása 72
37.8 Szálloda admin: foglalások kezelése (áttekintés) 72
Tipikus lépések 72
37.9 Szálloda admin: számlázási/cégadatok megadása 72

Lépések 72
37.10 Super admin: kezelőfunkciók (áttekintés) 73
38. Űrlapok használata 73
38.1 Gyakori mezők és jelentésük 73
38.2 Hibakezelés űrlapoknál 73
39. Hibaüzenetek és problémák 74
39.1 Hibás bejelentkezés 74
39.2 E-mail nincs megerősítve 74
39.3 Hiányzó adatok / hibás formátum 74
39.4 Fizetési oldal hiba 74
40. Kijelentkezés 75
Lépések 75
41. Gyakori kérdések (GYIK) 75
41.1 Nem kapom meg a megerősítő e-mailt. Mit tegyek? 75
41.2 Mikor kapok check-in QR kódot? 75
41.3 Banki átutalásnál miért nem jön azonnal a QR kód? 75
41.4 Elfelejtettem a jelszavam. Hogyan tudom visszaállítani? 75
42. Rendszer rövid áttekintés 77
43. Hardverkomponensek 77
43.1. Központi egység (kártya tároló szekrény) 77
43.1.1. Használt eszközök 77
43.1.2. Működése 77
43.2 Kártyaolvasó 78
43.2.1. Használt eszközök 78
43.2.2 PCB 78
43.2.2 Működés 79
Felhasznált források és eszközök: 81
Programozási nyelvek és Keretrendszerek 81
Adatbázis-kezelés 81
Frontend technológiák és Könyvtárak 81

Fejlesztői eszközök és Verziókezelés 81
Tesztelés és Minőségbiztosítás 82
Biztonság és Hitelesítés 82
Hardver és Speciális technológiák 82
Dokumentációk és Szabványok 82
Mesterséges Intelligencia (AI) eszközök 83

Projekt specifikáció

1. Bevezetés

1.1 A feladat rövid ismertetése

HotelFlow – Egy online platform, ahol a vendég megtalálhatja álmai szállását, a szállodák pedig gondoskodhatnak arról, hogy vendégeik kényelmesen és gördülékenyen foglalhassanak, miközben az adminisztrációs folyamatokat is megkönnyíti. A szállodáknak megkönnyíti a foglalások kezelését és egyszerűvé teszi a számlák kiállítását.

Vendégoldali funkciók

- Szálláskeresés: A felhasználó város, dátum és létszám megadásával listát kap az elérhető szálláshelyekről.
- Szűrés és böngészés: Az ajánlatok szűrhetők (ár, szolgáltatások, szoba típusa alapján), valamint részletes szállodai leírások és képek segítik a döntést.
- Bejelentkezés: Amennyiben a felhasználó foglalni szeretne, a rendszer automatikusan bejelentkezést kér. Amennyiben nincs regisztrált fiók, lehetőség van a regisztrációra.
- Egyszerű foglalás: A kiválasztott szállás néhány kattintással lefoglalható. A rendszer rögzíti a foglalás adatait és a vendég visszaigazolást kap a foglalási kérvény rögzítéséről.
- A felhasználó megerősítést kap, amikor a szálloda visszaigazolta a foglalást.
- Fizetés szimuláció: Valódi online fizetési rendszer integrálása nem történik meg, helyette a folyamat szimulált formában jelenik meg.
- Foglalás kezelése: A felhasználók belépve megtekinthetik és nyomon követhetik saját foglalásaikat.

Szállodai oldali funkciók

- Szobák és árak kezelése: A szállásadók rögzíthetik a szobatípusokat, árakat és a hozzájuk tartozó szolgáltatásokat.
- Foglalások nyomon követése: A beérkező foglalások megjelennek az adminisztrációs felületen, így könnyen áttekinthető a foglaltság.
- Egyszerű adminisztráció: A rendszer segít a foglalások rendezett tárolásában, alapvető nyilvántartás vezetésében.
- Alapvető kommunikáció: A foglalást a szálloda igazolja vissza, amiről a felhasználó értesítést kap.

Előnyök és célok

- Vendégeknek: gyors és egyszerű keresés, áttekinthető kínálat, kényelmes foglalási folyamat.

- Szállodáknak: könnyen kezelhető adminisztrációs felület, áttekinthető foglaláskezelés.

2. Elvárások

2.1 Platform és működés

A HotelFlow teljes egészében webes megoldás, vagyis nincs szükség külön program telepítésére. Egy modern böngészőben (telefonon, tableten, számítógépen) is kényelmesen működik. A háttérben egy megbízható adatbázis gondoskodik arról, hogy a foglalások és adatok mindig biztonságban legyenek, a kapcsolat pedig titkosítva zajlik.

2.2 Használt technológiák

A rendszer modern, bevált technológiákra épül:

- **Laravel:** a szerver oldali logikáért felel, kezeli az adatokat és a folyamatokat.
- **Vue.js:** a felhasználói felület épül rá, így a rendszer látványos és könnyen kezelhető.
- **MySQL:** adatbázis, ahol minden foglalás és felhasználói adat tárolódik.
- **Sanctum:** hozzáférések kezelése és titkosítása.
- **Vite:** A frontend gyors rendereléséért felel

2.3 Mit kell átadni

A projekt végeredményeként egy teljesen működő rendszer jön létre: a forráskód, a dokumentáció és a használati útmutató mind az átadott csomag részét képezi..

2.4 Fejlesztési szabályok

Fontos a tiszta, átlátható kód, a megfelelő tesztelés, valamint a megfelelő dokumentáció. Csapatban dolgozunk, a változtatásokat verziókezelőben követjük (GitHub).

2.5 Modulok

A rendszer több részből épül fel:

- **Vendég felület:** keresés, foglalás, fizetés.
- **Szállodai admin:** szobák, árak, foglalások és számlázás kezelése.
- **Központi admin:** a szállodák és felhasználók ellenőrzése, rendszer beállításai.
- **RFID alapú beléptetés, QR kódos check-in:** A vendégek gyors, érintésmentes és biztonságos beléptetése a szálláshelyen.

2.6 Fejlesztési lehetőségek

Az általunk megálmodott rendszer teljes mértékben skálázható és számos fejlesztési lehetőség rejlik benne:

- **Zárt chatszolgáltatás:** A szállásadó és a vendég közötti zárt chat alkalmazás integrálása.

3. A rendszer működése

3.1 Vendégélmény

A vendég belép a HotelFlow oldalára, ahol egy egyszerű keresőmező fogadja. Néhány adat megadásával – város, tartózkodás dátum tól-ig, létszám – rögtön megjelennek a találatok. A szálláshely oldalán képek, leírás és árak várják. A felhasználó kiválasztja a megfelelő szállás által kínált szobát, illetve az esetleges plusz szolgáltatásokat. A foglalás elkezdésénél a rendszer bejelentkezést kér. Amennyiben nincs regisztrált fiókja a felhasználónak, akkor lehetősége van regisztrálni egyet. A foglalás után azonnal érkezik a visszaigazolás, valamint a számla, és ha szükséges, e-mail értesítés. A foglalásokat a szállásadónak viszsza kell igazolni, és az elkészült számlát a felhasználó a felhasználói fiókjában tudja elérni, amiről emailes értesítést kap.

3.2 Szállodai adminisztráció

A szálloda dolgozói egy külön adminisztrációs felületet kapnak, ahol felvihetik a szobákat, beállíthatják az árakat, kezelhetik a foglalásokat. A rendszer automatikusan generálja a számlákat. A szálloda el tudja fogadni a bejövő foglalásokat, és a rendszer automatikusan könyveli azt a szálloda foglalási naptárába.

3.3 Központi adminisztráció

Egy központi adminisztrátor jogosultsággal rendelkező, gondoskodik arról, hogy csak valódi szállodák és felhasználók kerüljenek a rendszerbe. Ők kezelik a vitás ügyeket, és ellenőrzik a szolgáltatók hitelességét. Az összes adathoz hozzáférnek.

3.4 Foglalás és fizetés folyamata

1. A vendég kiválasztja a szállást.
2. Ha nincs bejelentkezve, akkor bejelentkezik.
3. Kitölti az adatokat, majd a kiválasztott fizetési móddal fizet.
4. A rendszer visszaigazolja a foglalást.
5. A szállásadó elfogadja a foglalást, és a rendszer automatikusan kiállítja a számlát.

Ezzel az öt lépéssel sokkal egyszerűbb a szállások kezelése.

4. Nem funkcionális követelmények

A rendszer nemcsak funkcióiban, hanem minőségében is magas szintet képvisel:

- Gyors és megbízható (a válaszidő mindig rövid).
- Mindig elérhető (évi 99,9%-os rendelkezésre állás).
- Betartja az adatvédelmi szabályokat (GDPR).
- Rugalmasan bővíthető a jövőben.

5. Dokumentáció

A projekt része lesz minden olyan leírás, amely segíti a rendszer használatát és üzemeltetését:

- **Felhasználói útmutató:** a vendégek és szállodák részére.
- **Forráskód dokumentáció:** könnyebb karbantartásért.

6. Összegzés

A **HotelFlow** egy olyan rendszer, amely egyesíti a modern szállásfoglaló oldalak kényelmét a magyar környezethez igazított funkciókkal. A vendégek gyorsan és egyszerűen találnak szállást, a szállodák pedig könnyen kezelhetik mindennapi működésüket.

7. Projekt adatlap

- Projekt neve: Hotelflow
- Feladat rövid ismertetése: Egy szállodai foglalási rendszer, fejlett szolgáltatói adminisztrátori lehetőségekkel.
- Felhasznált programozási nyelvek: Vue.js, SQL, Laravel, C++, Python
- Specifikációt összeállította: Monostori Márk György, Szabó Máté

Adatbázis specifikáció

8. Bevezetés

Ez a dokumentum a HotelFlow rendszer adatbázisának technikai specifikáció-ját tartalmazza.

A célja, hogy egy fejlesztő számára egyértelműen bemutassa az adatbázis szerkezetét, a táblák közötti kapcsolatokat, valamint az egyes entitásokhoz tartozó mezőket és kulcsokat.

A dokumentum alapján az adatbázis szerkezete könnyen rekonstruálható és implementálható bármely relációs adatbázis-kezelő rendszerben (pl. PostgreSQL, MySQL, SQLite).

9. Táblák

9.1 users

A rendszer összes felhasználóját tárolja (vendég, hotel tulajdonos, admin).

- Attribútumok:
  - id – Elsődleges kulcs (PK), egyedi azonosító
  - name – Felhasználó teljes neve
  - email – Egyedi e-mail cím (UNIQUE)
  - password – Titkosított jelszó
  - isVerified – Email hitelesítés állapota
  - role – Felhasználói szerepkör
  - tax_number – Adószám
  - bank_account – Bankszámlaszám
  - eu_tax_number – EU közösségi adószám
  - two_factor_secret – 2FA titkos kulcs
  - two_factor_enabled – Kétfaktoros hitelesítés állapota
  - email_verification_token – Email megerősítő token
  - email_verified_at – Email hitelesítés időpontja
  - created_at – Rekord létrehozás ideje
  - updated_at – Rekord módosítás ideje
- Kapcsolatok:
  - 1:N → password_reset_tokens
  - 1:N → sessions
  - 1:N → two_factor_recovery_tokens
  - 1:N → hotels
  - 1:N → serviceTags

- 1:N → bookings
- 1:N → booking_payments (confirmed_by_user_id)

9. 2 password_reset_tokens

Jelszó-visszaállító tokeneket tárol.

- Attribútumok:
  - id – Elsődleges kulcs (PK)
  - user_id – Kapcsolódó felhasználó (FK → users.id)
  - token – Egyedi visszaállító token (UNIQUE)
  - expires_at – Token lejárati idő
  - created_at – Létrehozás ideje
  - updated_at – Módosítás ideje
- Kapcsolatok:
  - N:1 → users

9.3 sessions

Bejelentkezési munkameneteket tárol.

- Attribútumok:
  - id – Elsődleges kulcs (PK)
  - user_id – Kapcsolódó felhasználó (FK → users.id)
  - session_token – Egyedi session token (UNIQUE)
  - expires_at – Lejárati idő
  - created_at – Létrehozás ideje
  - updated_at – Módosítás ideje
- Kapcsolatok:
  - N:1 → users

9.4 two_factor_recovery_tokens

Kétfaktoros hitelesítés recovery tokenjeit tárolja.

- Attribútumok:
  - id – Elsődleges kulcs (PK)
  - user_id – Kapcsolódó felhasználó (FK → users.id)
  - token_hash – Titkosított recovery token (UNIQUE)
  - expires_at – Lejárati idő
  - used_at – Felhasználás időpontja
  - created_at – Létrehozás ideje
  - updated_at – Módosítás ideje

- Kapcsolatok:
  - N:1 → users

9.5 hotels

A rendszerben regisztrált szállodák alapadatait tárolja.

- Attribútumok:
  - id – Elsődleges kulcs (PK)
  - user_id – Tulajdonos felhasználó (FK → users.id)
  - name – Hotel neve
  - location – Földrajzi elhelyezkedés
  - description – Részletes leírás
  - type – Szállás típusa
  - starRating – Csillag besorolás
  - is_approved – Admin jóváhagyás állapota
  - cover_image – Borítókép elérési út
  - tax_number – Hotel adószám
  - bank_account – Hotel bankszámlaszám
  - eu_tax_number – EU adószám
  - createdAt – Létrehozás ideje
- Kapcsolatok:
  - N:1 → users
  - 1:N → rooms
  - 1:N → services
  - 1:N → bookings
  - 1:N → rfidKeys
  - 1:1 → devices
  - 1:N → pending_handshakes
  - M:N → serviceTags (hotelTagRelation)

9.6 rooms

Szobák adatai.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - hotels_id (bigint, FK, NOT NULL)
  - name (varchar, NOT NULL)
  - description (text)
  - pricePerNight (int, NOT NULL)

    - capacity (int, NOT NULL)
    - basePrice (decimal(10,2), NOT NULL)
    - createdAt (timestamp)
- Kapcsolatok:
    - N:1 → hotels
    - 1:N → imagesRelation
    - 1:N → bookingsRelation
    - 1:N → roomTagRelation
    - 1:N → rfidKeyConnection
    - 1:N → rfid_assignments

9.7 images

Képek tárolása.

- Attribútumok:
    - id (bigint, PK, AUTO_INCREMENT)
    - url (varchar, NOT NULL) – Kép elérési út
- Kapcsolatok:
    - 1:N → imagesRelation

9.8 imagesRelation

Szoba–kép kapcsolótábla.

- Attribútumok:
    - images_id (bigint, PK, FK)
    - rooms_id (bigint, PK, FK)
- Kapcsolatok:
    - N:1 → images
    - N:1 → rooms

9.9 services

Extra szolgáltatások.

- Attribútumok:
    - id (bigint, PK, AUTO_INCREMENT)
    - name (varchar, NOT NULL)
    - description (text)
    - price (int, NOT NULL)
    - hotels_id (bigint, FK)
- Kapcsolatok:
    - N:1 → hotels
    - 1:N → servicesRelation

9.10 serviceTags

Címkék.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - user_id (bigint, FK)
  - name (varchar, NOT NULL)
- Kapcsolatok:
  - N:1 → users
  - 1:N → hotelTagRelation
  - 1:N → roomTagRelation

9.11 hotelTagRelation

Hotel–tag kapcsolótábla.

- Attribútumok:
  - hotels_id (bigint, PK, FK)
  - serviceTags_id (bigint, PK, FK)
- Kapcsolatok:
  - N:1 → hotels
  - N:1 → serviceTags

9.12 roomTagRelation

Szoba–tag kapcsolótábla.

- Attribútumok:
  - rooms_id (bigint, PK, FK)
  - serviceTags_id (bigint, PK, FK)
- Kapcsolatok:
  - N:1 → rooms
  - N:1 → serviceTags

9.13 bookings

Foglalások.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - users_id (bigint, FK, NOT NULL)
  - hotels_id (bigint, FK, NOT NULL)
  - startDate (date, NOT NULL)
  - endDate (date, NOT NULL)
  - totalPrice (int, NOT NULL)
  - checkInToken (varchar)

    - checkInstatus (varchar)
    - checkInTime (timestamp)
    - checkOutTime (timestamp)
    - status (varchar, DEFAULT "pending")
    - createdAt (timestamp)
- Kapcsolatok:
    - N:1 → users
    - N:1 → hotels
    - 1:N → bookingsRelation
    - 1:N → guests
    - 1:N → reviews
    - 1:1 → invoices
    - 1:1 → booking_payments
    - 1:1 → booking_invoice_details
    - 1:N → rfid_assignments
    - 1:N → servicesRelation

9.14 bookingsRelation

Foglalás–szoba kapcsolótábla.

- Attribútumok:
    - booking_id (bigint, PK, FK)
    - rooms_id (bigint, PK, FK)
- Kapcsolatok:
    - N:1 → bookings
    - N:1 → rooms

9.15 guests

Foglaláshoz tartozó vendégek.

- Attribútumok:
    - id (bigint, PK, AUTO_INCREMENT)
    - idNumber (varchar, NOT NULL)
    - name (varchar, NOT NULL)
    - dateOfBirth (date, NOT NULL)
    - bookings_id (bigint, FK, NOT NULL)
    - createdAt (timestamp)
- Kapcsolatok:
    - N:1 → bookings

9.16 invoices

Számlák.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - booking_id (bigint, FK, NOT NULL)
  - invoice_number (varchar, UNIQUE, NOT NULL)
  - status (varchar, DEFAULT "draft")
  - subtotal (decimal(10,2), NOT NULL)
  - tax_amount (decimal(10,2), DEFAULT 0)
  - total_amount (decimal(10,2), NOT NULL)
  - tax_rate (int, DEFAULT 27)
  - issue_date (date, NOT NULL)
  - due_date (date, NOT NULL)
  - pdf_path (varchar)
  - payment_token (varchar, UNIQUE)
  - approved_at (timestamp)
  - sent_at (timestamp)
  - created_at (timestamp)
  - updated_at (timestamp)
- Kapcsolatok:
  - N:1 → bookings

9.17 booking_payments

Fizetések.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - booking_id (bigint, UNIQUE, FK, NOT NULL)
  - method (varchar, DEFAULT "bank_transfer")
  - status (varchar, DEFAULT "pending")
  - confirmed_at (timestamp)
  - confirmed_by_user_id (bigint, FK)
  - created_at (timestamp)
  - updated_at (timestamp)
- Kapcsolatok:
  - N:1 → bookings
  - N:1 → users

9.18 booking_invoice_details

Számlázási adatok.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - booking_id (bigint, UNIQUE, FK, NOT NULL)
  - customer_type (varchar, DEFAULT "private")
  - full_name (varchar, NOT NULL)
  - email (varchar, NOT NULL)
  - company_name (varchar)
  - tax_number (varchar)
  - country (varchar)
  - city (varchar)
  - postal_code (varchar)
  - address_line (varchar)
  - note (text)
  - created_at (timestamp)
  - updated_at (timestamp)
- Kapcsolatok:
  - N:1 → bookings

9.19 servicesRelation

Foglalás–szolgáltatás kapcsolótábla.

- Attribútumok:
  - services_id (bigint, PK, FK)
  - bookings_id (bigint, PK, FK)
- Kapcsolatok:
  - N:1 → services
  - N:1 → bookings

9.20 rfidKeys

RFID kártyák.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - hotels_id (bigint, FK, NOT NULL)
  - isUsed (boolean, DEFAULT false)
  - rfidKey (varchar, UNIQUE, NOT NULL)
  - name (varchar)
  - type (varchar)
  - label (varchar)
  - status (varchar)

    - created_at (timestamp)
    - updated_at (timestamp)
- Kapcsolatok:
    - N:1 → hotels
    - 1:N → rfid_assignments
    - 1:N → rfidKeyConnection

9.21 rfidKeyConnection

RFID–szoba kapcsolat.

- Attribútumok:
    - id (bigint, PK, AUTO_INCREMENT)
    - rfidKeys_id (varchar, FK, NOT NULL)
    - rooms_id (bigint, FK, NOT NULL)
- Kapcsolatok:
    - N:1 → rfidKeys (rfidKey mező alapján)
    - N:1 → rooms

9.22 rfid_assignments

RFID hozzárendelések foglalásokhoz és szobához.

- Attribútumok:
    - id (bigint, PK, AUTO_INCREMENT)
    - rfid_key_id (bigint, FK, NOT NULL)
    - booking_id (bigint, FK)
    - room_id (bigint, FK, NOT NULL)
    - reserved_from (date)
    - reserved_to (date)
    - assigned_at (timestamp)
    - released_at (timestamp)
    - created_at (timestamp)
    - updated_at (timestamp)
- Kapcsolatok:
    - N:1 → rfidKeys
    - N:1 → bookings
    - N:1 → rooms

9.23 devices

Hotelhez tartozó hardver eszközök.

- Attribútumok:
    - id (bigint, PK, AUTO_INCREMENT)
    - name (varchar)

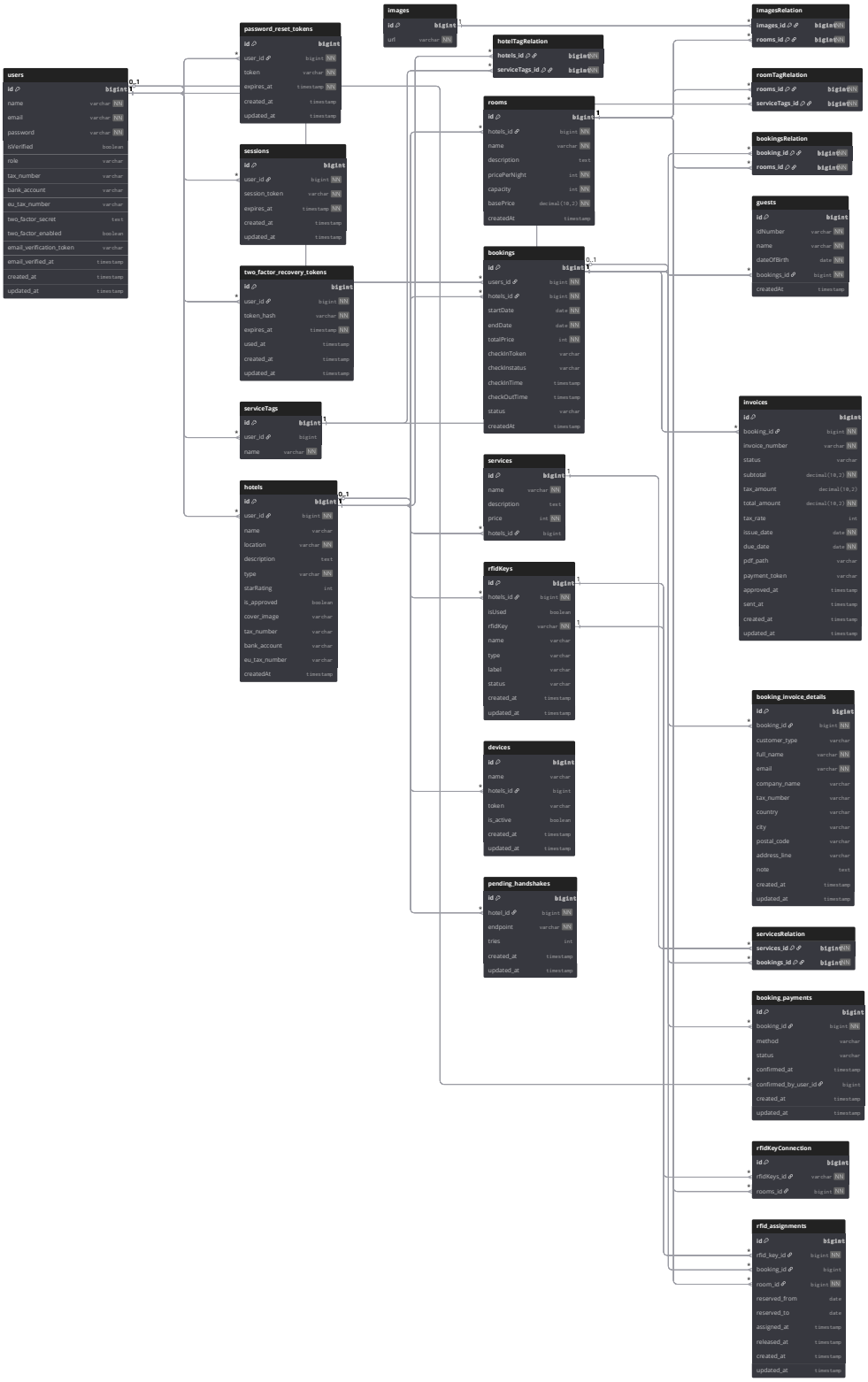
  - hotels_id (bigint, UNIQUE, FK)
  - token (varchar, UNIQUE)
  - is_active (boolean, DEFAULT false)
  - created_at (timestamp)
  - updated_at (timestamp)
- Kapcsolatok:
  - N:1 → hotels

9.24 pending_handshakes

Függőben lévő eszköz kommunikációk.

- Attribútumok:
  - id (bigint, PK, AUTO_INCREMENT)
  - hotel_id (bigint, FK, NOT NULL)
  - endpoint (varchar, NOT NULL)
  - tries (int, DEFAULT 0)
  - created_at (timestamp)
  - updated_at (timestamp)
- Kapcsolatok:
- N:1 → hotels

9.25 Adatbázis EK diagram



Algoritmus dokumentáció

10. Bevezetés

Ez a dokumentum a HotelFlow rendszer speciális algoritmusainak a leírását tartalmazza.

Minden időpont a backendben UTC-ben tárolódik; a frontend felelős az átalakításért a lokális megjelenítéshez. A dokumentum alapján a backend könnyen rekonstruálható és fejleszthető bármely erre alkalmas keretrendszerben. (pl.: Django, Laravel, Node.js, ASP.NET)

11. Bejelentkezés (Login)

- **Cél**: Felhasználó azonosítása és token generálása.
- **UI Interakció:**
    1. Felhasználó a login oldalon látja az email és jelszó mezőket, valamint a Bejelentkezés gombot.
    2. Kitölti a mezőket és rákattint a gombra.
    3. Hiba esetén (üres mező, rossz jelszó) a mezők felett hibaüzenet jelenik meg: „Érvénytelen email vagy jelszó”.
    4. Sikeres bejelentkezés esetén:
    5. A felhasználót átirányítja a dashboardra vagy kezdőoldalra.
    6. A token tárolódik a frontenden (localStorage / cookie) a további API hívásokhoz.
- Bemenet: email, password
- Kimenet: access_token, token_type
- UI kimenet: hibaüzenet vagy dashboard megjelenítése
- **Eljárás:**
    1. A rendszer ellenőrzi, hogy a Request tartalmazza az email és password mezőket.
    2. Ha bármelyik hiányzik → hibaüzenet visszaadása.
    3. Kikeresi az adatbázisból (users tábla) az adott email-hez tartozó felhasználót.
    4. Ha nincs ilyen user → hibás hitelesítés.
    5. Ellenőrzi a jelszó egyezőségét a tárolt hash (passwordHash) és a beérkező jelszó között.
    6. Ha nem egyezik → hibás hitelesítés.
    7. Ha minden rendben van:
    8. Generál egy API token-t a felhasználónak (Sanctum createToken).
    9. Visszaadja a tokent és a felhasználó azonosító adatait JSON formátumban.

12. Regisztráció (Register)

- **Cél:** Új felhasználó létrehozása.
- **UI Interakció:**
    1. A regisztrációs oldalon kiválasztja a felhasználó, hogy szállodaként vagy vendégként szeretne regisztrálni.
    2. Felhasználó a regisztrációs oldalon látja a mezőket: **név**, **email**, **jelszó**, **szálloda esetén a szállodának az adatait.**
    3. Kitöltés után a **Regisztráció** gombbal küldi el az adatokat.
    4. Hibák:
        - Üres mező → mező mellett piros hibaüzenet
        - Már létező email → „Ez az email már foglalt”
    5. Sikeres regisztráció után:
    6. Megjelenik a „Regisztráció sikeres!” üzenet.
    7. Felhasználót átirányíthatjuk a **login oldalra**.
- **Bemenet:** name, email, password, role, location-hotel esetén, type-hotel esetén
  Kimenet: user objektum, message
  UI kimenet: sikerüzenet + átirányítás loginra
- **Eljárás**:
    1. Ellenőrzi, hogy minden kötelező mező (name, email, password, role) jelen van és valid.
    2. Ellenőrzi, hogy az email nem foglalt az adatbázisban.
    3. Ha minden valid:
    4. A jelszót hash-eli (Hash::make)
    5. Létrehoz egy új User rekordot az adatbázisban.
    6. Visszaadja az új felhasználó adatait (ID, név, email, role, stb.).

13. Szállásfoglalás (Booking)

Cél: Felhasználó szobát foglal a szállodában.

UI Interakció:

- Felhasználó a **szállodai oldal** listájából kiválasztja a kívánt hotelt.
- Megnyitja a hotel részleteit: **szobák, árak, képek, szolgáltatások**.
- Kiválasztja a szobákat és a foglalás dátumát.
- Opció: extra szolgáltatások kiválasztása (pl. reggeli, wellness).
- Foglalás gomb: Foglalás → AJAX POST request az API-hoz.
- Hibák:
  1. Lejárt dátum, nem elérhető szoba → üzenet a felhasználónak
- Sikeres foglalás:
  1. Megjelenik a „Foglalás sikeres!” üzenet

2. Átirányítás a **saját foglalások oldalra** vagy megjelenítés modalban
3. Check-in token később a foglalás részleteinél látható

Bemenet: user_id, rooms_id, startDate, endDate, services
Kimenet: booking objektum, status
UI kimenet: sikerüzenet, foglalás részletek, navigáció a saját foglalásokhoz

- **Eljárás:**
  1. Ellenőrzi a beérkező foglalási adatokat (dátumok, szobák, felhasználó létezése).
  2. Létrehoz egy Booking rekordot az bookings táblában.
  3. Létrehozza a kapcsolót a szobákhoz a bookingsRelation táblában.
  4. Ha tartalmaz szolgáltatásokat → kapcsolja a servicesRelation táblához.
  5. Generálhat check-in és check-out tokeneket a szobákhoz.
  6. Foglalás státusza kezdetben pending.
  7. Visszaadja a foglalás adatait JSON formátumban.

14. Szállodai oldal – foglalás visszaigazolása

- **Cél:** Szálloda visszaigazolja vagy elutasítja a foglalást.
- **UI Interakció:**
  1. Szállodai admin a **foglalások listája** oldalon látja az összes pending státuszú foglalást.
  2. A foglalás részletei tartalmazzák: **vendég neve, dátumok, szoba, extra szolgáltatások**.
  3. Gombok: **Visszaigazolás**, **Mégse**.
  4. Klikk után AJAX request a confirm vagy cancel action-re.
  5. Visszaigazolás után:
  6. A státusz a listában **confirmed** vagy **cancelled** lesz.
  7. Értesítés a felhasználónak e-mailben.
  8. Hiba esetén a felületen **piros hibaüzenet** jelenik meg.

Bemenet: booking_id, action
Kimenet: booking status
UI kimenet: státusz frissítés, üzenet a felhasználónak

- **Eljárás:**
  1. Ellenőrzi, hogy a foglalás létezik (bookings tábla).
  2. Ha confirm → állítja a foglalás státuszát confirmed.
  3. Ha cancel → állítja a státuszt cancelled.
  4. Frissíti a foglalás dátumát (updatedAt) és esetleg e-mail értesítést küld a felhasználónak.

5. A foglalás státuszát a frontenden és mobil appban is lekérdezhetjük.

15. Kártyás beléptető rendszer (QR kód és RFID)

Cél: A vendégek be és kiléptetés, szállodai recepciós alkalmazása nélkül. Egyszerűen és gyorsan.

Eljárás:

1. A vendég érkezéskor leolvassa az emailben kapott QR kódot
2. A fő egység a saját adatbázisából megkeresi a foglalás azonosítót, ami a QR kódban volt.
3. A foglalásban kiosztott szobákhoz tartozó kártyákat az egység kiadja
4. A vendég a kártyával tud belépni a szoba ajtaján, ami e kártya adatait MQTT protokollon keresztül juttatja el a főegységen futó ellenőrző szoftvernek. Ha helyes a kártya száma, akkor ugyan ezzel a metódussal kapja vissza a terminál és nyitja a zárat.
5. Kilépéskor a belépéshez használt QR kóddal tudja leadni a kártyát.

Logikai elv: Terminál → MQTT → Ellenőrző program → MQTT → Terminál

16. Szoba felvétele a szállodához

- **Cél:** Szálloda adminisztrátor hozzáad egy új szobát a hotelhez.
- **UI Interakció:**
  1. Szállodai admin a **Hotel kezelése** oldalon látja a **Szobák** listáját.
  2. Gomb: **Új szoba hozzáadása**
  3. **Form**:
     - Név (pl. „Deluxe szoba”)
     - Leírás
     - Ár/éj
     - Kapacitás
     - Alapár
  4. Képek feltöltése (opcionális)
  5. Kitöltés után **Mentés** gomb:
     - Backend ellenőrzi a kötelező mezőket
     - Létrehoz egy új rooms rekordot
     - Ha vannak képek, azokat feltölti az images táblába és kapcsolja a imagesRelation-nel
     - Visszaadja az új szoba adatait

- **Bemenet:** hotel_id, name, description, pricePerNight, capacity, basePrice, images
  Kimenet: szoba objektum, képek listája
  UI kimenet: sikerüzenet, szoba listában frissítés

17. Szálloda adatbővítése / szerkesztése

- **Cél:** Szálloda adminisztrátor frissíti a hotel adatait, képeket ad hozzá, ratingeket, szolgáltatásokat.
- **UI Interakció:**
  1. Szállodai admin a **Hotel szerkesztése** oldalon látja a mezőket:
     1. Név
     2. Leírás
     3. Típus (hotel, apartman, villa…)
     4. Csillagbesorolás
     5. Képek kezelése (feltöltés, törlés)
     6. Szolgáltatások hozzáadása / törlése
     7. Cím / lokáció
  2. Admin módosítja az adatokat, feltölti képeket.
  3. Mentés gomb:
     - Backend ellenőrzi az inputot
     - Frissíti a hotels rekordot
     - Új képek → images tábla + imagesRelation
     - Szolgáltatások → services + servicesRelation
  4. Sikeres mentés után:
     - Visszajelzés: „Hotel adatai frissítve”
     - A hotel részletei a frontenden azonnal frissülnek
- **Bemenet:** hotel_id, name, description, type, starRating, location, images, services
  Kimenet: hotel objektum, frissített képek és szolgáltatások
  UI kimenet: sikerüzenet, frissített hotel oldalon megjelenés

18. Szoba elérhetőségének ellenőrzése

- **Cél:**
  - Megállapítani, hogy egy szoba a megadott időintervallumban (startDate – endDate) **szabad-e**, vagy már létezik hozzá ütköző foglalás.
- **Bemenet:**
  - room_id – a választott szoba azonosítója
  - startDate – foglalás kezdete
  - endDate – foglalás vége

- **Kimenet:**
  - Szoba foglalható / nem foglalható (boolean)
  - Ütköző foglalások listája (opcionális)
- **Eljárás leírása:**

  A rendszer megkapja a szoba azonosítóját és a kért időszakot.

  1. Az adatbázisban megkeresi az adott szobához tartozó foglalásokat (bookingsRelation + bookings).
  2. A foglalások közül azokat vizsgálja, amelyek:
     - státusza **confirmed** vagy **pending**,
     - **átfednek** az új foglalás időszakával.
  3. Két foglalás akkor fed át, ha teljesül az alábbi feltétel:
     - existing.startDate <= new.endDate ÉS existing.endDate >= new.startDate
  4. Ha talál a rendszer akár egyetlen ütköző foglalást:
  5. az eredmény: **szoba nem foglalható**.
  6. Ha nincs ütközés:
     - az eredmény: **szoba szabad és foglalható**.
  7. **Kivételes esetek**
     - Ha a szoba nem létezik → hibaüzenet (404)
     - Ha a dátumok hibásak → hiba (400)

19. Foglalás árkalkulációja

- **Cél:**
  - A foglalás teljes árának kiszámítása a kiválasztott szobák, azok kapacitása, az ott tartózkodó vendégek száma, a foglalás időtartama és az igénybe vett szolgáltatások alapján.
- **Bemenet:**
  - rooms[] – szobák ID-listája
  - services[] – szolgáltatások ID-listája
  - startDate, endDate – foglalási időszak
  - guests[] – vendégek listája (név, idNumber, dateOfBirth)
- **Kimenet:**
  - Teljes ár (integer)
  - Részletes bontás (szobaár, szolgáltatásár, napok száma)
- **Eljárás leírása:**
  - A rendszer megkapja a foglaláshoz tartozó szobák listáját.
  - Meghatározza a foglalás hosszát napokban:
    - napok = endDate - startDate
  - A rendszer betölti az összes érintett szoba adatait:
    - basePrice

        - pricePerNight
        - capacity
    - Betölti a vendégek számát:
        - vendégekSzáma = guests tömb hossza
    - Minden szobánál kiszámolja a szoba árát:
        - Szoba árképzés logikája:
            - A szoba ára = (basePrice × vendégek száma) + (pricePerNight × napok száma)
            - Ha több szoba van, a rendszer összeadja az egyes szobákra számolt árakat.
    - Ha a felhasználó igénybe vesz szolgáltatásokat:
        - betölti a services táblából az árakat,
        - összeadja őket:
            - szolgáltatások_ára = összeg(services.price)
    - A teljes ár:
        - totalPrice = összesSzobaÁr + szolgáltatások_ára
    - A rendszer a kalkulált értéket elmenti a bookings.totalPrice mezőbe.
- **Kivételes esetek:**
    - Ha a dátumok hibásak → hiba (400)
    - Ha vendégek száma meghaladja a szobák összkapacitását → hiba (409)
    - Ha egy szoba nem létezik → hiba (404)

Tesztdokumentáció

20. A tesztelés célja

A tesztelés fő célja annak biztosítása, hogy a rendszer megfelelően működjön, a felhasználói felület intuitív legyen, az API végpontok helyesen működjenek, és a biztonsági mechanizmusok megfelelően védjék a rendszert. A tesztelés során a következő területeket vizsgáljuk:

- Funkcionális tesztelés: legfontosabb funkciók helyes működése
- Biztonsági tesztelés: hitelesítés, autorizáció, adatvédelem
- API tesztelés: REST API végpontok helyes működése
- Felhasználói felület tesztelés: UI/UX elemek helyes működése
- Hibakezelés: hibás bemenetek és hibaüzenetek kezelése

21. Tesztkörnyezet

21.1. Hardver környezet

Komponens	Specifikáció
Processzor	Intel Core i5 vagy ekvivalens
Memória	Minimum 8 GB RAM
Tárhely	Minimum 10 GB szabad hely
Hálózat	Stabil internetkapcsolat

21.2. Szoftver környezet

Komponens	Verzió/Megnevezés
Operációs rendszer	Windows 10/11, Linux, macOS
Backend framework	Laravel 12.x
Frontend framework	Vue.js 3.x
Build eszköz	Vite
Adatbázis	MySQL
PHP verzió	PHP 8.2+
Node.js	Node.js 18+
Composer	2.x
NPM	9.x+

21.3. Böngészők

A tesztelés során a következő böngészőkben végeztük a teszteket:

- Google Chrome (legfrissebb verzió)
- Mozilla Firefox (legfrissebb verzió)
- Microsoft Edge (legfrissebb verzió)

21.4. Backend futtatási környezet

A backend szerver a következő környezetben fut:

- Laravel fejlesztői szerver (php artisan serve –host=0.0.0.0)
- Port: 8000 (alapértelmezett), a windows tűzfalon ki -és bemenő szabályoknál engedélyezve
- API alap URL: http:// (a szerver gép aktuális IP címe):8000/api
- CORS beállítások engedélyezve
- Laravel Sanctum token-alapú hitelesítés

21.5. Frontend környezet

- A frontend szerver a következő környezetben fut:
- Vue.js fejlesztési szerver (npm run dev)
- Port: 3000 (szintén engedélyezve a tűzfalon)
- API: VITE_BASE_URL-el beállítva a backend szerver alap url-jére

22. Tesztelési módszerek

22.1. Manuális tesztelés

A manuális tesztelés során a tesztelő közvetlenül használja a felhasználói felületet, és ellenőrzi a funkciók helyes működését. Ez a módszer lehetővé teszi a felhasználói élmény értékelését és a vizuális hibák azonosítását.

22.2 Automatizált tesztelés (Selenium)

Az automatizált tesztelés során Selenium framework-ot használunk, amivel hatékonyabbá tesszük a tesztelés folyamatát. A következőket foglalja magában:

- Bejelentkezés
- Keresés
- Navigáció tesztelése
- Adminisztrátori funkció tesztelése (Összetett űrlap)

22.3. API tesztelés

Az API tesztelés során a REST API végpontokat teszteljük, ellenőrizzük a válaszokat, a státusz kódokat és az adatstruktúrákat. Swagger UI-t használunk az API dokumentáció megtekintéséhez és a végpontok teszteléséhez.

23. Tesztesetek

Az alábbi táblázatokban részletesen dokumentáltuk a főbb teszteseteket. Minden teszteset tartalmazza a teszt azonosítóját, a funkció nevét, a teszt leírását, a bemeneti adatokat, a várt eredményt, a tényleges eredményt és a teszt státuszát.

23.1. Frontend tesztesetek

Teszt ID	Funkció neve	Teszt leírás	Bemeneti adatok	Várt eredmény	Tényleges eredmény	Státusz
FT-01	**Bejelentkezés alap felhasználóként**	A bejelentkezés tesztelése természetes felhasználók számára.	Email: szabo.mate@diak.szbi-pg.hu Jelszó: Gum55NDx /login oldalon	Bejelentkezés után átirányítás a /booking oldalra	Bejelentkezés után átirányítás a /booking oldalra	Passed

```
async function loginFullFlowTest() {
  const driver = await new Builder().forBrowser("chrome").build();
  try {
    // Login oldal
    await driver.get("http://172.16.6.12:3000/login");
    const emailInput = await driver.wait(
      until.elementLocated(By.id("email")),
      10000
    );
    // Kitöltés
    await emailInput.sendKeys("szabo.mate@diak.szbi-pg.hu");
    const passwordInput = await driver.findElement(By.id("password"));
    await passwordInput.sendKeys("Gum55NDx");
    // Submit
    await driver.findElement(By.css("button[type='submit']")).click();
    const skipButton = await driver.wait(
      until.elementLocated(By.css(".btn-skip")),
      100000
    );
    // Biztos ami biztos látható is legyen
    await driver.wait(until.elementIsVisible(skipButton), 5000);
    // Kattintás
    await skipButton.click();
    await driver.wait(
      until.urlIs("http://172.16.6.12:3000/bookings"),
      10000
    );
    console.log("✅ Teljes login flow PASS");
```

FT-02	**Bejelentkezés alap felhasználóként (hibakezelés tesztelése)**	A bejelentkezés tesztelése természetes felhasználók számára hibás felhasználói adatokkal.	Email: wrong@test.com Jelszó: wrongpassword /login oldalon	Bejelentkezés után űrlap törlése, a felhasználó marad a /login oldalon	Bejelentkezés után űrlap törlése, a felhasználó marad a /login oldalon	Passed

```
async function loginFailRedirectTest() {
  const driver = await new Builder().forBrowser("chrome").build();

  try {
    // Login oldal
    await driver.get("http://172.16.13.18:3000/login");

    // Email és jelszó rosszul
    const emailInput = await driver.wait(until.elementLocated(By.id("email")), 5000);
    await emailInput.sendKeys("wrong@test.com");

    const passwordInput = await driver.findElement(By.id("password"));
    await passwordInput.sendKeys("wrongpassword");

    // Submit
    await driver.findElement(By.css("button[type='submit']")).click();

    // Rövid várakozás, hogy a router feldolgozza
    await driver.sleep(1000); // 1 másodperc

    // Ellenőrizzük az URL-t
    const currentUrl = await driver.getCurrentUrl();

    if (currentUrl.endsWith("/login")) {
      console.log("✅ Hibás login teszt PASS - maradtunk a /login oldalon");
    } else {
      console.log("❌ Hibás login teszt FAIL - átirányították:", currentUrl);
    }
```

FT-03	**Oldalnavigáció tesztelése**	A felső navigációs menüben rákattintunk a Regisztráció menüpontra	**-**	Kattintás után a frontend router megfelelően átirányít mindket a /register oldalra	Kattintás után a frontend router megfelelően átirányít mindket a /register oldalra	Passed

```
async function navigationTest() {
  const driver = await new Builder().forBrowser("chrome").build();

  try {
    // --- Megnyitjuk a főoldalt ---
    await driver.get("http://172.16.13.18:3000/");

    // --- Kattintás a "Regisztráció" gombra ---
    const signupBtn = await driver.wait(
      until.elementLocated(By.css(".btn-signup")),
      5000
    );
    await signupBtn.click();


    // --- Várjuk a /signup oldalra való átirányítást ---
    await driver.wait(
      until.urlIs("http://172.16.13.18:3000/register"),
      5000
    );
    console.log("✅ Navigációs teszt PASS - sikeres átirányítás a /register oldalra");

  } catch (err) {
    console.log("❌ Navigációs teszt FAIL");
    console.error(err);
  } finally {
    await driver.quit();
  }
}
```

FT-04	**Keresés mező tesztelése**	Megadjuk a foglalás helyét, az érkezés és a távozás dátumát majd rákattintunk a keresés gombr**a**	Lista random eleme a városok közül, és a jelenlegi dátumhoz képest 2 hónappal következő 2 egymást követő nap	A keresés után eltűnnek az ajánlott szállások és megjelennek a keresés eredményei	A keresés után eltűnnek az ajánlott szállások és megjelennek a keresés eredményei	Passed
FT-05	**Keresés mező tesztelése hibás adatokkal**	Megadjuk a foglalás helyét, az érkezés és a távozás dátumát (az érkezés dátuma később van mint a távozásé) majd rákattintunk a keresés gombra	Lista random eleme a városok közül, és a jelenlegi dátumhoz képest 2 hónappal következő 2 egymást követő nap (az érkezés később van mint a távozás)	A rendszer nem engedi betáplálni a hamarabbi dátumot a távozásnál, mint amit az érkezésnél megadtunk, figyelmeztető üzenet jelenik meg	A rendszer nem engedi betáplálni a hamarabbi dátumot a távozásnál, mint amit az érkezésnél megadtunk, figyelmeztető üzenet jelenik meg	Passed
FT-06	**Szolgáltatás hozzáadása szállodához hotel adminisztrátorként**	Bejelentkezés után kétlépcsős azonosítást manuálisan elvégezzük, majd az admin dashboardon megadunk egy szolgáltatást a már meglévők mellé, majd elküldjük az űrlapot.	Név: Wifi szolgáltatás Leírás: Stabil internet Ár: 0 Ft	A megfelelő adatok megadását követően a rendszerben regisztrálódik az új szolgáltatás.	A megfelelő adatok megadását követően a rendszerben regisztrálódik az új szolgáltatás	Passed
FT-07	**Szolgáltatás hozzáadása szállodához hotel adminisztrátorként (hiányos űrlap)**	Bejelentkezés után kétlépcsős azonosítást manuálisan elvégezzük, majd az admin dashboardon megadunk egy szolgáltatást a már meglévők mellé, majd elküldjük az űrlapot.	Név: - Leírás: - Ár: 0 Ft	Az űrlap elküldése után hibaablak jelenik meg, ami megadja az első olyan mező nevét ami üres. A rendszerben hiányos adatokkal nem regisztrálódik szolgáltatás.	Az űrlap elküldése után hibaablak jelenik meg, ami megadja az első olyan mező nevét ami üres. A rendszerben hiányos adatokkal nem regisztrálódik szolgáltatás.	Passed

23.2. API tesztesetek

A védett végpontok teszteléséhez megfelelő szintű bearer token szükséges, amit hozzá kell adni a Swagger UI autentikáció részéhez. Ilyen tesztek:

- API-02 (user)
- API-03 (user)
- API-04 (hotel)
- API-06 (hotel)
- API-07 (hotel)

Teszt ID	Funkció neve	Teszt leírás	Várt eredmény	Státusz
API-01	Bejelentkezés vendég felhasználóknak	Bejelentkezés alap felhasználóknak, két lépcsős azonosítás nélkül	Felhasználói adatok, illetve bearer token visszaadása	Passed
Bemeneti adatok: *POST /api/auth/login* *{* *"email": "szabo.mate@diak.szbi-pg.hu",* *"password": "Gum55NDx"* *}*			**Tényleges eredmény:** *{* *"id": 73,* *"name": "Szabó Máté",* *"email": "szabo.mate@diak.szbi-pg.hu",* *"role": "user",* *"two_factor_enabled": 0,* *"created_at": "2026-02-24T07:16:46.000000Z",* *"token":* *"28|ZOowDKIqf9cEFXR6Cckc7WMsKMNxkNW9pPlbGqHB1373311e",* *"show_2fa_prompt": true* *}*	
API-02	A vendéghez tartozó foglalások lekérdezése	A megfelelő bearer tokent csatolva GET kérést intézünk az alábbi végpontra, amelyen várjuk a foglalások részleteit.	A foglalások részletei, hozzá tartozó szolgáltatások és szobák adatai.	Passed
Bemeneti adatok: *GET /api//bookings/user/{73}*			**Tényleges eredmény:** *{* *"bookings": [* *{* *"id": 144,* *"users_id": 73,* *"hotels_id": 1,* *"startDate": "2026-02-25",* *"endDate": "2026-02-26",* *"totalPrice": 20000,* *"checkInToken": "8iZo5mC4wGMVWo47",* *"checkInstatus": null,* *"checkInTime": null,* *"checkOutTime": null,* *"status": "pending",* *"createdAt": "2026-02-25 13:25:09",*	

```
    "rooms": [
      {
        "id": 1,
        "hotels_id": 1,
        "name": "Standard Room",
        "description": "Kényelmes, klasszikus szoba két fő részére.",
        "pricePerNight": 10000,
        "capacity": 2,
        "basePrice": "10000.00",
        "createdAt": "2026-02-24 08:16:46",
        "pivot": {
          "booking_id": 144,
          "rooms_id": 1
        }
      }
    ],
    "guests": [],
    "services": [
      {
        "id": 116,
        "name": "WiFi szolgáltatás",
        "description": "Teszt leírás",
        "price": 0,
        "hotels_id": 1,
        "pivot": {
          "bookings_id": 144,
          "services_id": 116
        }
      },
      {
        "id": 119,
        "name": "WiFi Szolgáltatás",
        "description": "Stabil internet.",
        "price": 0,
        "hotels_id": 1,
        "pivot": {
          "bookings_id": 144,
          "services_id": 119
        }
      }
    ]
  }
]
}
```


API-03	**Új foglalás létrehozása**	Új szállodai foglalás rögzítése a kiválasztott szobákkal, vendégszámmal és extra szolgáltatásokkal.	Sikeres foglalás visszaigazolása a foglalási adatokkal és a végösszeggel.	Passed

Bemeneti adatok:


```
POST /api/bookings
{
  "hotelId": 1,
  "startDate": "2026-03-10",
  "endDate": "2026-03-12",
  "rooms": [
    {
      "id": 1,
```


Tényleges eredmény:


```
{
  "id": 145,
  "user_id": 73,
  "hotel_id": 1,
  "start_date": "2026-03-10",
  "end_date": "2026-03-12",
  "status": "pending",
  "total_price": 20000,
```

```
      "guests": 2
    }
  ],
  "services": [
    116,
    119
  ],
  "payment_method": "credit_card",
  "invoice_details": {
    "billing_name": "Szabó Máté",
    "billing_address": "6000 Kecskemét, Teszt utca 1."
  }
}
```

```
  "rooms": [
    {
      "id": 1,
      "name": "Standard Room"
    }
  ],
  "services": [
    116,
    119
  ]
}
```


API-04	**Foglalás státuszának frissítése**	Egy meglévő foglalás állapotának módosítása (pl. függőben lévőről visszaigazoltra) a foglalás ID-ja alapján.	A rendszer frissíti a státuszt, és visszaküldi a teljes foglalási objektumot a módosított adatokkal és a kapcsolódó entitásokkal.	Passed

Bemeneti adatok:


```
PUT /bookings/update/{id}
{
  "status": "confirmed"
}
```


Tényleges eredmény:


```
"message": "Foglalás sikeresen frissítve",
  "booking": {
    "id": 144,
    "users_id": 73,
    "hotels_id": 1,
    "startDate": "2026-02-25",
    "endDate": "2026-02-26",
    "totalPrice": 20000,
    "checkInToken": "8iZo5mC4wGMVWo47",
    "checkInstatus": null,
    "checkInTime": null,
    "checkOutTime": null,
    "status": "confirmed",
    "createdAt": "2026-02-25 13:25:09",
    "user": {
      "id": 73,
      "name": "Szabó Máté",
      "email": "szabo.mate@diak.szbi-pg.hu",
      "tax_number": null,
      "bank_account": null,
      "eu_tax_number": null,
      "two_factor_secret": null,
      "two_factor_enabled": 0,
      "password": "$2y$12$yr2J74U7KJNwzjMSYPKXV.Rk5AoEqORExSENhWaUTWEY3flCz3tYG",
      "isVerified": 1,
      "email_verification_token": null,
      "email_verified_at": "2026-02-25 12:40:19",
      "role": "user",
      "created_at": "2026-02-24T07:16:46.000000Z",
      "updated_at": "2026-02-25T11:40:19.000000Z"
    },
    "rooms": [
      {
        "id": 1,
        "hotels_id": 1,
        "name": "Standard Room",
        "description": "Kényelmes, klasszikus szoba két fő részére.",
```

```
      "pricePerNight": 10000,
      "capacity": 2,
      "basePrice": "10000.00",
      "createdAt": "2026-02-24 08:16:46",
      "pivot": {
        "booking_id": 144,
        "rooms_id": 1
      }
    }
  ],
  "services": [
    {
      "id": 116,
      "name": "WiFi szolgáltatás",
      "description": "Teszt leírás",
      "price": 0,
      "hotels_id": 1,
      "pivot": {
        "bookings_id": 144,
        "services_id": 116
      }
    },
    {
      "id": 119,
      "name": "WiFi Szolgáltatás",
      "description": "Stabil internet.",
      "price": 0,
      "hotels_id": 1,
      "pivot": {
        "bookings_id": 144,
        "services_id": 119
      }
    }
  ],
  "guests": [],
  "payment": {
    "id": 1,
    "booking_id": 144,
    "method": "bank_transfer",
    "status": "pending",
    "confirmed_at": null,
    "confirmed_by_user_id": null,
    "created_at": "2026-02-25T12:25:09.000000Z",
    "updated_at": "2026-02-25T12:25:09.000000Z"
  }
}
}
```


API-05	**Szállodák és ajánlatok keresése**	Szállodák keresése város, időintervallum és vendégszám alapján, a hozzájuk tartozó szobatípusokkal és árajánlatokkal (plans).	A feltételeknek megfelelő szállodák listája, benne a szobák képeivel, szolgáltatásokkal és kategorizált ajánlatokkal (pl. legolcsóbb)	Passed

Bemeneti adatok:	**Tényleges eredmény:**
GET /api/search?city=Budapest&startDate=2026-08-28&endDate=2026-08-29&guests=2	*{ "hotel_id": 1, "name": "Hotel Budapest", "location": "Budapest, Hungary", "type": "hotel",*

```
  "starRating": 4,
  "description": "Egy szép hotel a város szívében.",
  "services": [
    { "id": 116, "name": "WiFi szolgáltatás", "price": 0 },
    { "id": 119, "name": "WiFi Szolgáltatás", "price": 0 }
  ],
  "plans": [
    {
      "room_count": 1,
      "total_price": 10000,
      "label": "Legolcsóbb",
      "rooms": [
        {
          "room_id": 1,
          "name": "Standard Room",
          "capacity": 2,
          "price": 10000
        }
      ]
    },
    {
      "room_count": 1,
      "total_price": 15000,
      "label": "Alternatíva",
      "rooms": [
        {
          "room_id": 2,
          "name": "Deluxe Room",
          "capacity": 3,
          "price": 15000
        }
      ]
    }
  ]
}
]
```


API-06	**Új szoba hozzáadása szállodához**	Új szoba létrehozása egy adott szállodához. A művelethez **hotel-admin** szerepkör és érvényes bearer token szükséges.	A szoba sikeresen létrejön, visszakapjuk a generált ID-t, részleteket és a hozzárendelt képlistát.	Passed

Bemeneti adatok:

POST /api/rooms/create/1


```
{
  "name": "Executive Suite",
  "hotels_id": 1,
  "description": "Luxus lakosztály panorámás kilátással és saját jakuzzival.",
  "capacity": 2,
  "pricePerNight": 250,
  "basePrice": 80,
  "price": 45000
}
```


Tényleges eredmény:


```
{
  "hotels_id": 1,
  "name": "Executive Suite",
  "description": "Luxus lakosztály panorámás kilátással és saját jakuzzival.",
  "pricePerNight": 250,
  "capacity": 2,
  "basePrice": 80,
  "createdAt": "2026-02-25T13:21:58.717705Z",
  "id": 173
}
```

API-07	**Szoba törlése**	Egy konkrét szoba végleges eltávolítása a rendszerből az azonosítója (ID) alapján.	A szoba sikeresen törlődik, a válaszban pedig megerősítés érkezik a műveletről.	Passed
Bemeneti adatok: *DELETE /api/rooms/delete/173*		**Tényleges eredmény:** *{* *"message": "Room deleted successfully"* *}*		

24. Hibakezelés értékelése

A rendszer hibakezelése kritikus fontosságú a felhasználói élmény és a rendszer stabilitása szempontjából. Az alábbiakban értékeljük a rendszer hibakezelési képességeit.

24.1. Validációs hibák kezelése

A rendszer megfelelően kezeli a validációs hibákat. Minden bemeneti mező validálva van, és a hibák érthető, felhasználóbarát formában jelennek meg. A backend Laravel validációs szabályokat használ, míg a frontend Vue.js validációs logikát implementál.

- Email formátum ellenőrzése: érvénytelen email cím esetén hibaüzenet
- Kötelező mezők ellenőrzése: hiányzó kötelező mezők esetén validációs hiba
- Dátum validáció: múltbeli dátumok és érvénytelen dátum tartományok kezelése
- Szám validáció: negatív számok és nem numerikus értékek kezelése
- Jelszó validáció: minimális hosszúság és komplexitás követelmények
- Egyedi értékek ellenőrzése: duplikált email címek és egyéb egyedi mezők kezelése

24.2. API hibakezelés

Az API végpontok megfelelő HTTP státusz kódokat adnak vissza különböző hibás helyzetekben:

- 400 Bad Request: érvénytelen kérés formátum
- 401 Unauthorized: hiányzó vagy érvénytelen token
- 403 Forbidden: nincs jogosultság az erőforráshoz
- 404 Not Found: nem létező erőforrás
- 422 Unprocessable Entity: validációs hibák
- 500 Internal Server Error: szerver oldali hibák

25.3. Frontend hibakezelés

A frontend alkalmazás megfelelően kezeli a hibákat és felhasználóbarát hibaüzeneteket jelenít meg. A hibaüzenetek magyar nyelven jelennek meg, és segítik a felhasználót a probléma megoldásában.

- Hálózati hibák kezelése: API hívások sikertelensége esetén értelmes hibaüzenet
- Timeout kezelés: hosszú válaszidők esetén timeout kezelés
- Form validáció: valós idejű validáció a felhasználói bemeneteknél
- Hibaüzenetek megjelenítése: Toast üzenetek vagy inline hibaüzenetek
- Loading állapotok: betöltési animációk hosszú műveleteknél

25. Alapvető biztonsági tesztelés

25.1. Szerepkör-alapú hozzáférés-vezérlés

A rendszer szerepkör-alapú hozzáférés-vezérlést (RBAC) használ. A következő szerepkörök vannak definiálva:

Szerepkör	Leírás	Jogosultságok
user	Vendég felhasználó	Foglalások létrehozása, saját foglalások kezelése, profil kezelés
hotel	Szálloda adminisztrátor	Szálloda kezelés, szobák kezelése, foglalások kezelése, szolgáltatások kezelése
super_admin	Super adminisztrátor	Teljes rendszer hozzáférés, minden entitás kezelése

A tesztelés során ellenőriztük, hogy minden szerepkör csak az őt megillető erőforrásokhoz fér hozzá:

- User szerepkör nem fér hozzá hotel admin funkciókhoz
- Hotel admin szerepkör nem fér hozzá más szállodák adataihoz
- Super admin szerepkör hozzáfér minden erőforráshoz
- Jogosulatlan hozzáférési kísérletek megfelelően elutasítva (403 Forbidden)

25.2. Token-alapú hitelesítés

A rendszer Laravel Sanctum token-alapú hitelesítést használ. A tokenek a következőképpen működnek:

- Token generálás: sikeres bejelentkezés után token generálása
- Token validáció: minden védett végpont ellenőrzi a token érvényességét

- Token lejárat: tokenek lejárati idejük után érvénytelenné válnak
- Token törlés: kijelentkezéskor a token törlődik
- Token hiánya: token nélküli kérések 401 Unauthorized választ kapnak

25.3. Jogosulatlan hozzáférési kísérletek kezelése

A rendszer megfelelően kezeli a jogosulatlan hozzáférési kísérleteket:

- Más felhasználó foglalásainak elérése: 403 Forbidden
- Más szálloda adatainak elérése: 403 Forbidden
- Admin funkciók elérése user szerepkörrel: 403 Forbidden
- Érvénytelen token használata: 401 Unauthorized
- Lejárt token használata: 401 Unauthorized

25.4. Adatvédelem

A rendszer megfelelően védi a felhasználói adatokat:

- Jelszavak hash-elve tárolódnak (bcrypt)

26. További fejlesztési javaslatok

A jövőbeli fejlesztésekhez az alábbi javaslatokat teszünk:

- Automatizált tesztelés bevezetése: Unit tesztek, Integration tesztek, E2E tesztek
- Teljesítmény tesztelés: Load testing, stress testing nagy terhelés alatt
- Biztonsági audit: Részletes biztonsági audit végrehajtása
- Monitoring és logging: Részletesebb naplózás és monitoring rendszer
- Böngésző kompatibilitás: További böngészőkben való tesztelés

27. Következtetés

A HotelFlow szálloda foglalási rendszer tesztelése alapján elmondható, hogy a rendszer megfelelően működik és készen áll a használatra. A főbb funkcionalitások helyesen implementálva vannak, a biztonsági mechanizmusok megfelelően működnek, és a hibakezelés átfogó. A rendszer stabil, és a felhasználói felület intuitív és könnyen használható.

A tesztelés során azonosított kisebb problémák nem akadályozzák a rendszer használatát, de javasolt ezek javítása a jövőbeli fejlesztések során. A rendszer továbbfejlesztésének javasolt irányai a teljesítmény optimalizálás, az automatizált tesztelés bevezetése, és a dokumentáció bővítése.

Frontend és Backend dokumentáció

28. Rendszer architektúra

28.1 Általános felépítés

A rendszer kliens–szerver architektúrára épül: a böngészőben futó kliens (frontend) HTTP-n keresztül hívja a szerveroldali REST API-t (backend). A backend hitelesítést és jogosultságkezelést végez, üzleti logikát futtat, majd adatbázisban perzisztál.

28.2 Fő komponensek

- Frontend (Vue 3 + Vite): felhasználói felület, űrlapok, navigáció, API hívások.
- Backend (Laravel 12): API, hitelesítés (Sanctum), számla PDF generálás, e-mail küldés.
- Adatbázis (MySQL): üzleti adatok (felhasználók, szállodák, szobák, foglalások, számlák, RFID).

28.2.1 Kliens–szerver működés (folyamat-ábra szövegesen)

1) A felhasználó a frontend felületen műveletet indít (pl. foglalás).

2) A frontend a `services/*` rétegen keresztül HTTP kérést küld a backend `/api/...` végpontjára.

3) A backend validál, auth-ot és role-t ellenőriz (`auth:sanctum`, `role:*`).

4) Üzleti logika fut (árkalkuláció, kapcsolótáblák mentése, PDF generálás, e-mail küldés).

5) A backend JSON választ ad, a frontend pedig frissíti a UI-t.

28.3 Frontend–backend kapcsolat

A frontend az `axios` könyvtárat használja. A `frontend/src/services/api.js` interceptor automatikusan csatolja a Bearer tokent az `Authorization` headerben, ha a felhasználó be van jelentkezve.

29. Felhasznált technológiák

29.1 Backend technológiák

- Nyelv: PHP 8.2+
- Framework: Laravel 12
- Adatbázis: MySQL 8 (fejlesztésben docker-compose)
- ORM: Eloquent ORM (modellek: `app/Models/*`)
- Hitelesítés: Laravel Sanctum (token alapú)
- PDF: dompdf (számla PDF)

- QR: endroid/qr-code + simple-qrcode (különböző QR felhasználások)

29.2 Frontend technológiák

- Framework: Vue 3
- Build tool: Vite
- Routing: Vue Router
- State: egyszerű reactive store (`src/stores/auth.js`)
- API kommunikáció: Axios (service layer: `src/services/*`)

29.3 Verziók / függőségek (a kódból)

A backend fő csomagjai a `backend/hotelflow_server/composer.json` alapján:

```
{
    "$schema":                          "https://getcomposer.org/schema.json",
    "name":                                                  "laravel/laravel",
    "type":                                                          "project",
    "description":  "The  skeleton  application  for  the  Laravel  framework.",
    "keywords":                    ["laravel",                    "framework"],
    "license":                                                            "MIT",
    "require":                                                               {
        "php":                                                        "^8.2",
        "dompdf/dompdf":                                              "^2.0",
        "endroid/qr-code":                                            "^5.0",
        "intervention/image":                                        "^3.11",
        "laravel/framework":                                         "^12.0",
        "laravel/sanctum":                                            "^4.2",
        "laravel/tinker":                                          "^2.10.1",
        "simplesoftwareio/simple-qrcode":                              "^4.2"
    },
    "require-dev":                                                           {
        "fakerphp/faker":                                            "^1.23",
        "laravel/pail":                                             "^1.2.2",
        "laravel/pint":                                              "^1.24",
        "laravel/sail":                                              "^1.41",
        "mockery/mockery":                                             "^1.6",
        "nunomaduro/collision":                                        "^8.6",
        "phpunit/phpunit":                                          "^11.5.3"
    },
    "autoload":                                                              {
        "psr-4":                                                             {
            "App\\":                                                  "app/",
            "Database\\Factories\\":                       "database/factories/",
            "Database\\Seeders\\":                            "database/seeders/"
        },
        "files":                                                             [
            "app/Helpers/NumberToWords.php",
            "app/Helpers/ImageResizer.php"
        ]
    },
    "autoload-dev":                                                          {
        "psr-4":                                                             {
            "Tests\\":                                              "tests/"
        }
    },
    "scripts":                                                               {
        "setup":                                                             [
            "composer                                              install",
            "@php -r \"file_exists('.env') || copy('.env.example', '.env');\"",
            "@php                     artisan                    key:generate",
            "@php             artisan             migrate             --force",
            "npm                                                   install",
            "npm                           run                        build"
        ],
```

```
        "dev":                                                                  [
            "Composer\\Config::disableProcessTimeout",
            "npx concurrently -c \"#93c5fd,#c4b5fd,#fb7185,#fdba74\" \"php arti-
san serve\" \"php artisan queue:listen --tries=1\" \"php artisan pail --ti-
meout=0\" \"npm run dev\" --names=server,queue,logs,vite --kill-others"
        ],
        "test":                                                                 [
            "@php artisan config:clear --ansi",
            "@php artisan test"
        ],
        "post-autoload-dump":                                                   [
```

A frontend fő függőségei a `frontend/package.json` alapján:

```
{
  "name":                                                   "hotelflow-frontend",
  "version":                                                              "1.0.0",
  "type":                                                                 "module",
  "scripts":                                                                     {
    "dev":                                                                  "vite",
    "build":                                                  "vite build",
    "preview":                                                 "vite preview"
  },
  "dependencies":                                                                {
    "axios":                                                              "^1.6.0",
    "chromedriver":                                                     "^145.0.6",
    "selenium-webdriver":                                                "^4.41.0",
    "vue":                                                                "^3.4.0",
    "vue-router":                                                         "^4.2.5"
  },
  "devDependencies":                                                             {
    "@vitejs/plugin-vue":                                                 "^5.2.4",
    "vite":                                                               "^6.4.0"
  }
}
```

30. Backend dokumentáció

30.1 Backend projekt struktúra

Gyökér: `backend/hotelflow_server` (Laravel alkalmazás).

Fontos mappák:

- app/Http/Controllers – API végpontok megvalósítása (kontrollerek)
- app/Models – Eloquent modellek/entitások
- app/Mail – e-mail sablonokat összefogó Mailables osztályok
- resources/views – Blade nézetek (email, PDF számla)
- routes/api.php – API route definíciók
- database/migrations – adatbázis séma változásai

30.2 Adatbázis felépítése

A projekt tartalmaz egy dbdiagram.io kompatibilis DBML állományt (`database_documentation.dbml`), amely a táblákat, mezőket és kapcsolatokat leírja. A rendszer fő táblacsoportjai:

- Felhasználók és hitelesítés (users, password_reset_tokens, two_factor_recovery_tokens)
- Szállodák/szobák/szolgáltatások (hotels, rooms, services, képek és tag kapcsolatok)
- Foglalások (bookings + kapcsolótáblák)
- Számlázás és fizetés (invoices, booking_payments, booking_invoice_details)
- RFID és eszközök (rfid_keys, assignments, connections, devices…)

4.2.1 Táblák összefoglalója (teljes lista)

Tábla	Leírás (Note)
users	Felhasználók táblája. Típusok: user, hotel, super_admin
password_reset_tokens	Jelszó visszaállítási tokenek
sessions	Felhasználói munkamenetek
two_factor_recovery_tokens	Kétfaktoros autentikáció helyreállítási tokenek (SHA256 hash)
hotels	Szállodák táblája. is_approved: super admin jóváhagyása szükséges a megjelenítéshez
rooms	Szobák táblája
images	Képek táblája

imagesRelation	Szobák és képek kapcsolata (many-to-many)
services	Szolgáltatások táblája. hotels_id null esetén globális szolgáltatás
serviceTags	Szolgáltatás címkék. Használható szállodákhoz és szobákhoz
hotelTagRelation	Szállodák és címkék kapcsolata (many-to-many)
roomTagRelation	Szobák és címkék kapcsolata (many-to-many)
servicesRelation	Foglalások és szolgáltatások kapcsolata (many-to-many)
bookings	Foglalások táblája
bookingsRelation	Foglalások és szobák kapcsolata (many-to-many)
guests	Vendégek adatai foglalásonként
reviews	Vendég értékelések foglalásokhoz
invoices	Számlák táblája. payment_token: bankkártyás fizetéshez használt egyedi token
booking_payments	Foglalások fizetési információi. method: bank_transfer vagy card
booking_invoice_details	Foglalásonkénti számlázási adatok. customer_type: private vagy business
rfidKeys	RFID kulcsok táblája. type: guest vagy crew. status: aktuális állapot
rfidKeyConnection	RFID kulcsok és szobák közvetlen kapcsolata (régi rendszer)
rfid_assignments	RFID kulcs hozzárendelések. booking_id null esetén manuális hozzárendelés (pl. crew card). reserved_from/to: foglalás időszaka
devices	Regisztrált eszközök API hozzáféréshez. Egy szállodához egy eszköz
pending_handshakes	Függőben lévő eszköz handshake kérések
cache	Laravel cache tábla
cache_locks	Laravel cache lock tábla
jobs	Laravel queue jobs tábla

job_batches	Laravel job batches tábla
failed_jobs	Laravel failed jobs tábla
personal_access_tokens	Laravel Sanctum personal access tokens

Megjegyzés: a táblák mezőszintű részletezését a 6. fejezet adja meg (részletes mintával).

30.3 Modellek / entitások

A backend Eloquent modelleket használ. Példák a fő entitásokra és kapcsolataikra (a kódból):

- User: Sanctum tokenek, szerepkör (user/hotel/super_admin), e-mail verifikáció, 2FA mezők.
- Booking: user–hotel kapcsolat, rooms/services many-to-many, uests hasMany, invoice/payment hasOne.
- Invoice: booking-hoz tartozó számla, státusz (draft/approved/sent), PDF útvonal, payment_token.

30.4 API végpontok

Az API végpontok a `routes/api.php` fájlban vannak. Az alábbiakban csoportosítva dokumentáljuk őket.

4.4.1 Auth végpontok

HTTP	Végpont	Handler	Middleware
POST	/auth/register-user	AuthController@registerUser	
POST	/auth/register-hotel	AuthController@registerHotel	
POST	/auth/login	AuthController@login	
POST	/auth/logout	AuthController@logout	'auth:sanctum'
GET	/auth/verify-email/{token}	AuthController@verifyEmail	
POST	/auth/resend-verification	AuthController@resendVerificationEmail	
POST	/auth/forgot-password	AuthController@forgotPassword	
POST	/auth/reset-password	AuthController@resetPassword	

GET	/auth/user/{id}	AuthControl-ler@getUserById	'auth:sanctum'
GET	/auth/me	AuthControl-ler@me	'auth:sanctum'
PUT	/auth/updateu-ser/{id}	AuthControl-ler@updateUser	'auth:sanctum'
POST	/auth/deleteu-ser/{id}	AuthControl-ler@deleteUser	'auth:sanctum'
PUT	/auth/admin/profile	AuthControl-ler@updateU-serAdmin	'auth:sanctum', 'role:hotel'
POST	/auth/2fa/enable	AuthControl-ler@enable2FA	'auth:sanctum'
POST	/auth/2fa/verify-enable	AuthControl-ler@verifyAndE-nable2FA	'auth:sanctum'
POST	/auth/2fa/disable	AuthControl-ler@disable2FA	'auth:sanctum'
POST	/auth/verify-2fa	AuthControl-ler@verify2FA	'auth:sanctum'
POST	/auth/2fa/re-covery/request	AuthControl-ler@requ-estTwoFactorRe-covery	
POST	/auth/2fa/re-covery/confirm	AuthControl-ler@con-firmTwoFactorRe-covery	

30.4.2 Foglalás (bookings) végpontok

HTTP	Végpont	Handler	Middleware
POST	/bookings	BookingControl-ler@store	'auth:sanctum'
GET	/bookings/user/{userId}	BookingControl-ler@getBookings-ByUserId	'auth:sanctum'
GET	/bookings/hotel/{hote-lId}	BookingControl-ler@getBookings-ByHotelId	'auth:sanctum', 'role:hotel'

DELETE	/bookings/delete/{id}	BookingController@deleteBooking	'auth:sanctum'
PUT	/bookings/update-status/{id}	BookingController@updateStatus	'auth:sanctum'
PUT	/bookings/update/{id}	BookingController@update	'auth:sanctum', 'role:hotel'
POST	/bookings/{id}/confirm-payment	BookingController@confirmPayment	'auth:sanctum', 'role:hotel'
POST	/bookings/add-guest/{bookingId}	BookingController@addGuests	'auth:sanctum'
GET	/bookings	SuperAdminController@getAllBookings	
GET	/bookings/{id}	SuperAdminController@getBooking	
POST	/bookings	SuperAdminController@createBooking	
PUT	/bookings/{id}	SuperAdminController@updateBooking	
DELETE	/bookings/{id}	SuperAdminController@deleteBooking	

30.4.3 Számla / fizetés (invoices) végpontok

HTTP	Végpont	Handler	Middleware
GET	/invoices/booking/{bookingId}/preview	InvoiceController@generatePreview	
GET	/invoices/booking/{bookingId}	InvoiceController@getByBooking	
PUT	/invoices/{invoiceId}	InvoiceController@update	'role:hotel'
POST	/invoices/{invoiceId}/approve	InvoiceController@approve	
POST	/invoices/{invoiceId}/send	InvoiceController@send	

GET	/invoices/{invoiceId}/preview	InvoiceController@preview	
GET	/invoices/{invoiceId}/download	InvoiceController@download	
GET	/invoices/payment/{token}	InvoiceController@getByPaymentToken	
POST	/invoices/payment/{token}/process	InvoiceController@processPayment	
GET	/invoices	SuperAdminController@getAllInvoices	
GET	/invoices/{id}	SuperAdminController@getInvoice	
PUT	/invoices/{id}	SuperAdminController@updateInvoice	
DELETE	/invoices/{id}	SuperAdminController@deleteInvoice	

30.4.4 Szálloda (hotels) végpontok

HTTP	Végpont	Handler	Middleware
GET	/hotels	HotelController@getHotels	
GET	/hotels/{id}	HotelController@getHotelById	
POST	/hotels	HotelController@createHotel	'auth:sanctum', 'role:hotel'
PUT	/hotels/upgrade/{id}	HotelController@upgradeHotel	'auth:sanctum', 'role:hotel'
POST	/hotels/{id}/coverimage	HotelController@uploadCoverImage	'auth:sanctum', 'role:hotel'
DELETE	/hotels/{id}/coverimage	HotelController@deleteCoverImage	'auth:sanctum', 'role:hotel'
GET	/hotels/{id}/activities	HotelController@getRecentActivities	'auth:sanctum', 'role:hotel'

GET	/hotels/{id}/billing-info	HotelController@getHotelBillingInfo	'auth:sanctum', 'role:hotel'
PUT	/hotels/{id}/billing-info	HotelController@updateHotelBillingInfo	'auth:sanctum', 'role:hotel'
DELETE	/hotels/delete/{id}	HotelController@deleteHotel	'auth:sanctum', 'role:hotel'
GET	/hotels/all-with-rooms	HotelDataController@getAllHotelsWithRooms	
POST	/hotels/{hotel}/tags	HotelTagController@store	
DELETE	/hotels/{hotel}/tags/{tag}	HotelTagController@destroy	
GET	/hotels	SuperAdminController@getAllHotels	
GET	/hotels/{id}	SuperAdminController@getHotel	
POST	/hotels	SuperAdminController@createHotel	
PUT	/hotels/{id}	SuperAdminController@updateHotel	
POST	/hotels/{id}/approve	SuperAdminController@approveHotel	
DELETE	/hotels/{id}	SuperAdminController@deleteHotel	

30.4.5 Szobák (rooms) végpontok

HTTP	Végpont	Handler	Middleware
GET	/rooms/hotel/{hotel_id}	RoomController@getRoomsByHotelId	
GET	/rooms/{id}	RoomController@getRoomById	
POST	/rooms/create/{hotel_id}	RoomController@createRoom	'auth:sanctum', 'role:hotel'

DELETE	/rooms/delete/{id}	RoomController@deleteRoom	'auth:sanctum', 'role:hotel'
PUT	/rooms/update/{id}	RoomController@updateRoom	'auth:sanctum', 'role:hotel'
GET	/rooms/{roomId}/images	ImageController@roomImages	
POST	/rooms/{room}/tags	RoomTagController@store	
DELETE	/rooms/{room}/tags/{tag}	RoomTagController@destroy	
GET	/rooms	SuperAdminController@getAllRooms	
GET	/rooms/{id}	SuperAdminController@getRoom	
POST	/rooms	SuperAdminController@createRoom	
PUT	/rooms/{id}	SuperAdminController@updateRoom	
DELETE	/rooms/{id}	SuperAdminController@deleteRoom	

30.4.6 Szolgáltatások (services) végpontok

HTTP	Végpont	Handler	Middleware
GET	/services/hotel/{hotel_id}	ServiceController@getServicesByHotelId	
GET	/services/{id}	ServiceController@getServiceById	
POST	/services	ServiceController@createService	'auth:sanctum', 'role:hotel'
PUT	/services/{id}	ServiceController@updateService	'auth:sanctum', 'role:hotel'
DELETE	/services/{id}	ServiceController@deleteService	'auth:sanctum', 'role:hotel'

GET	/services	SuperAdminCont-roller@getAllSer-vices	
GET	/services/{id}	SuperAdminCont-roller@getService	
POST	/services	SuperAdminCont-roller@createSer-vice	
PUT	/services/{id}	SuperAdminCont-roller@upda-teService	
DELETE	/services/{id}	SuperAdminCont-roller@deleteSer-vice	

30.4.7 Címkék (tags) végpontok

HTTP	Végpont	Handler	Middleware
GET	/tags	ServiceTagCont-roller@index	
GET	/tags/usage	ServiceTagCont-roller@getUsage	
POST	/tags	ServiceTagCont-roller@store	'auth:sanctum', 'role:hotel'
PUT	/tags/{id}	ServiceTagCont-roller@update	'auth:sanctum', 'role:hotel'
DELETE	/tags/{id}	ServiceTagCont-roller@destroy	'auth:sanctum', 'role:hotel'

30.4.8 Képek (images) végpontok

HTTP	Végpont	Handler	Middleware
POST	/images	ImageControl-ler@store	'auth:sanctum', 'role:hotel'
POST	/images/link	ImageControl-ler@link	'auth:sanctum', 'role:hotel'
DELETE	/images/unlink	ImageControl-ler@unlink	'auth:sanctum', 'role:hotel'
DELETE	/images/{id}	ImageControl-ler@destroy	'auth:sanctum', 'role:hotel'
GET	/images/{id}	ImageControl-ler@show	

30.4.9 Keresés (search) és ajánlások (recommendations)

HTTP	Végpont	Handler	Middleware
GET	/search/locations	SearchControl-ler@getLocations	
GET	/search	SearchControl-ler@search-WithPlans	

30.4.10 RFID kulcsok

HTTP	Végpont	Handler	Middleware
GET	/rfid-keys	RFIDKeyControl-ler@index	
GET	/rfid-keys/bookings	RFIDKeyControl-ler@getAvailable-Bookings	
GET	/rfid-keys/calendar	RFIDKeyControl-ler@calendarAs-signments	
GET	/rfid-keys/assig-ned-rooms	RFIDKeyControl-ler@getRoomsAs-signedToCrew-Cards	
GET	/rfid-keys/{id}	RFIDKeyControl-ler@show	
GET	/rfid-keys/{id}/ma-nual-assignments	RFIDKeyControl-ler@manualAs-signments	
POST	/rfid-keys	RFIDKeyControl-ler@store	
PATCH	/rfid-keys/{id}	RFIDKeyControl-ler@update	
DELETE	/rfid-keys/{id}	RFIDKeyControl-ler@destroy	
POST	/rfid-keys/{id}/as-sign	RFIDKeyControl-ler@assign	
POST	/rfid-keys/{id}/as-sign-room	RFIDKeyControl-ler@as-signToRoom	
POST	/rfid-keys/{id}/rele-ase	RFIDKeyControl-ler@release	

DELETE	/rfid-keys/{id}/manual-assignments/{assignmentId}	RFIDKeyController@deleteManualAssignment	
GET	/rfid-keys	SuperAdminController@getAllRFIDKeys	
GET	/rfid-keys/{id}	SuperAdminController@getRFIDKey	
POST	/rfid-keys	SuperAdminController@createRFIDKey	
PUT	/rfid-keys/{id}	SuperAdminController@updateRFIDKey	
DELETE	/rfid-keys/{id}	SuperAdminController@deleteRFIDKey	

30.4.11 Eszközök

HTTP	Végpont	Handler	Middleware
GET	/devices/bookings/{hotelId}	DeviceController@getBookings	
PUT	/devices/update-booking/{bookingId}	DeviceController@updateData	
GET	/devices	SuperAdminController@getAllDevices	
GET	/devices/{id}	SuperAdminController@getDevice	
POST	/devices	SuperAdminController@createDevice	
PUT	/devices/{id}	SuperAdminController@updateDevice	
POST	/devices/{id}/regenerate-token	SuperAdminController@regenerateToken	

DELETE	/devices/{id}	SuperAdmin-Controller@dele-teDevice	

30.4.13 Példa kérés/válasz formátumok

Megjegyzés: a konkrét mezők endpointonként eltérnek; az alábbi minták a kódban megfigyelt általános JSON sémát mutatják.

```
POST /api/auth/login
Request JSON: { "email": "...", "password": "...", "two_factor_code"?: "123456"
}
Response JSON (siker): { "id": 1, "name": "...", "role": "user|hotel|super_ad-
min", "token": "..." }


POST /api/bookings
Request JSON (részlet): { userId, hotelId, startDate, endDate, rooms: [{id,
guests}], services?: [], payment_method?: "card|bank_transfer", invo-
ice_details?: {...} }
Response JSON: { "bookingId": 123, "totalPrice": 999 }
```

30.5 Hitelesítés és jogosultságkezelés

Bejelentkezés / token:

- A login végpont a felhasználó jelszavát ellenőrzi, majd Sanctum tokent generál. A token a frontendben `localStorage`-be kerül, és az axios interceptor automatikusan csatolja.

E-mail megerősítés:

- Regisztráció után a felhasználó e-mail verifikációs linket kap, és csak verifikáció után jelentkezhet be (kivétel: super_admin szerepkör).

2FA:

- A super_admin számára kötelező a 2FA, hotel adminoknál a frontend router guard kényszerítheti a bekapcsolást (a kódban szereplő szabály szerint).

Szerepkör ellenőrzés:

- A backend `role:*` middleware-t használ (lásd `app/Http/Middleware/RoleMiddleware.php`), amely a super_admin számára teljes hozzáférést enged, egyébként szerepkör-egyezést vár.

30.6.1 E-mail küldések áttekintése

A backend több ponton küld e-mailt (Mailables: `app/Mail/*`). Tipikus események:

- Regisztráció: e-mail verifikációs levél
- Jelszó visszaállítás: tokenes levél
- 2FA recovery: egyszer használatos helyreállítási link
- Foglalási kérelem elküldve: vendég értesítése + hotel admin értesítése
- Foglalás megerősítés: számla küldés (fizetési módtól függő sablon)
- Fizetés megerősítése után: check-in QR kód email

31. Frontend dokumentáció

31.1 Frontend projekt struktúra

Gyökér: `frontend/` (Vue 3 alkalmazás).

Fontos mappák:

- src/views – oldalak (vendég, admin, super admin)
- src/services – API szolgáltatások (axios wrapper)
- src/router – route definíciók + guardok
- src/stores – auth state (token + user)
- src/layouts – admin/super-admin layoutok (navigáció, sidebar)
- tests – Selenium alapú E2E tesztek

31.2 Komponensek

A projektben a fő, újrahasznosítható UI elemek a `src/components/` alatt vannak (pl. `DataTable`, `ConfirmDialog`, `Toast`, 2FA promptok, image upload).

31.3 Routing

A Vue Router útvonalai a `src/router/index.js` fájlban vannak. Meta mezők jelzik, hogy kell-e auth, és szükséges-e szerepkör. A `beforeEach` guard kezeli a beléptetést/átirányítást és a role alapú védelmet.

31.4 Állapotkezelés

Az auth állapotot a `src/stores/auth.js` kezeli egy `reactive` state-tel. Induláskor betölti a `localStorage`-ből a tokent és felhasználót. Login után elmenti őket, logoutnál törli.

31.5 API kommunikáció

Az API hívások a `src/services/*.js` fájlokban vannak. Az `src/services/api.js` axios instance beállítja a baseURL-t, és interceptorral hozzáadja a Bearer tokent. 401 esetén automatikus kijelentkeztetés történik.

31.6 Űrlapok és felhasználói interakciók

A validáció jelentős része backend oldali (Laravel `Request::validate`). Frontend oldalon a komponensek jellemzően egyszerű ellenőrzéseket végeznek (pl. kötelező mezők, dátumok), majd hiba esetén felhasználóbarát üzenetet jelenítenek meg.

31.7 Oldalak (views) funkcionális áttekintése

Vendég oldalak:

- /search – keresés és listázás
- /hotel/:id – szálloda részletek + foglalás indítása
- /bookings – saját foglalások
- /payment/:token – számla fizetés token alapján

Hotel admin oldalak:

- /admin – dashboard
- /admin/bookings – foglalások kezelése, számla műveletek
- /admin/rooms, /admin/services – erőforrás kezelés
- /admin/rfid-keys – RFID kulcsok kezelése

Super admin oldalak:

- /super-admin/dashboard – statisztikák
- /super-admin/users/hotels/rooms/services/bookings/invoices – CRUD jellegű admin funkciók
- /super-admin/devices – eszköz regisztrációk

32. Biztonsági megoldások

Hitelesítés:

- Sanctum token (Bearer) – védett végpontok `auth:sanctum` middleware-rel.
- Jelszavak hash-elése (`Hash::make`, `Hash::check`).
- E-mail verifikáció – bejelentkezés csak megerősített emaillel (nem super_admin).
- 2FA – super_admin kötelező, további szerepkörök opcionális.
- Jogosultságkezelés:
- Role middleware: `role:hotel`, `role:super_admin` stb.
- Erőforrás tulajdonlás ellenőrzés (pl. booking user_id/ hotel owner check).
- Input validáció:
- Laravel `validate` a kontrollerekben (dátumok, ID-k, enumok, max hossz).

33. Telepítés (Deployment)

33.2 Lokális fejlesztői futtatás

Backend: composer install, .env beállítás, artisan key:generate, migrate, artisan serve.

Frontend: npm install, Vite dev server.

33.3 Környezeti változók (összefoglaló)

A tesztelésnél használt adatbázis elérhető a projekt main branch-én.

Backend tipikus .env kulcsok:

```
APP_NAME=HotelFlow
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=szabomate403@gmail.com
MAIL_PASSWORD=zntwelklzpzudqqg
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=szabomate403@gmail.com
MAIL_FROM_NAME="Hotelflow"DB_CONNECTION=mysql
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=hotelflow
DB_USERNAME=root
DB_PASSWORD=mysql
```

Felhasználói dokumentáció

34. Rendszerkövetelmények

A rendszer webes alkalmazás, modern böngészőből használható.

Támogatott böngészők (ajánlott):

- Google Chrome (friss verzió)
- Microsoft Edge (friss verzió)
- Mozilla Firefox (friss verzió)
- Safari (macOS/iOS – friss verzió)

Internet kapcsolat: stabil, folyamatos kapcsolat javasolt (foglalás és fizetés során kiemelten).

Ajánlott eszközök:

- PC/laptop (admin felülethez különösen kényelmes)
- Mobiltelefon (vendég oldali böngészés/fizetés támogatott, reszponzív felületekkel)

35. Bejelentkezés a rendszerbe

35.1 Bejelentkezés lépései

- Nyissa meg a bejelentkezési oldalt (Bejelentkezés).
- Adja meg az e-mail címét.
- Adja meg a jelszavát.
- Kattintson a „BEJELENTKEZÉS” gombra.

35.2 Sikeres bejelentkezés után mi történik?

A rendszer a szerepkör alapján automatikusan a megfelelő felületre irányít:

Szerepkör	Hova irányít a rendszer?
Vendég (user)	„Foglalásaim” oldal
Szálloda admin (hotel)	Admin → Foglalások
Super admin (super_admin)	Super admin → Dashboard

35.3 Hibás adatok / tipikus problémák

Ha hibás e-mail/jelszó párost ad meg, a rendszer hibaüzenetet jelenít meg.

Ha a regisztráció után még nem erősítette meg az e-mail címét, a rendszer jelezni fogja, és felajánlhatja a megerősítő e-mail újraküldését.

35.4 Kétfaktoros hitelesítés (2FA)

Bizonyos fiókoknál a rendszer kérhet 2FA kódot. Ilyenkor az autentikátor alkalmazás (pl. Google Authenticator/Authy) által generált 6 számjegyű kódot kell megadni.

Ha elvész a telefon, a rendszer 2FA helyreállítást is kínál e-mailes linken keresztül.

36. A rendszer felépítése

36.1 Fő felületek (szerepkörök szerint)

Vendég (user) felületek:

- Keresés (szállások keresése)
- Szálloda részletek (csomagok, szobák, szolgáltatások)
- Foglalásaim (foglalások és vendégadatok kezelése, számla megtekintése)
- Fizetés (bankkártyás fizetési oldal a számla linken keresztül)
- Profil (adatmódosítás, 2FA, jelszó csere, fiók törlés)

Szálloda admin (hotel) felületek:

- Dashboard (statisztikák, legutóbbi tevékenységek, gyors műveletek)
- Foglalások (foglalások kezelése, számlázás és fizetés státuszok)
- Szobák / Szolgáltatások / Címkék / RFID kulcsok (kezelő oldalak)
- Cégadatok / Számlázási információk (adószám, bankszámlaszám, EU adószám)
- Profilom (fiókadatok, 2FA, jelszó, fiók törlés)

Super admin felületek:

- Dashboard (összesített statisztikák, legutóbbi foglalások)
- Felhasználók / Szállodák / Szobák / Szolgáltatások / Foglalások / Számlák / RFID / Eszközök

37. A rendszer fő funkciói

37.1 Regisztráció (Vendég vagy Szálloda admin)

Mire használható: új fiók létrehozása a rendszerben.

Hogyan érhető el: „Regisztráció” menüpont.

Lépések

- Válassza ki a szerepkört: „Vendég” vagy „Szálloda Admin”.
- Töltse ki a regisztrációs űrlapot (teljes név, e-mail, jelszó).
- Szálloda admin esetén adja meg a szálloda adatait (név, helyszín, típus, csillagok – opcionális).
- Kattintson a „REGISZTRÁLÁS” gombra.
- Ellenőrizze az e-mail fiókját, és kattintson a megerősítő linkre (e-mail cím megerősítése).

37.2 Bejelentkezés + 2FA

Mire használható: hozzáférés a személyes/admin felületekhez.

Lépések

- Adja meg az e-mail címet és jelszót.
- Ha a rendszer 2FA kódot kér, nyissa meg az autentikátor alkalmazást.
- Írja be a 6 számjegyű kódot, majd erősítse meg.

37.3 Szállás keresése (Vendég)

Mire használható: szállodák keresése város, dátum és vendégszám alapján.

Lépések

- Nyissa meg a „Keresés” oldalt.
- Adja meg: „Hová utazik?” (város/helyszín).
- Válassza ki az érkezés és távozás dátumát.
- Adja meg a vendégek számát (minimum 1).
- Kattintson a „Keresés” gombra.
- A találatoknál kattintson a kívánt szállodára a részletek megtekintéséhez.

Tipp: a rendszer ajánlott szállásokat is megjeleníthet a keresés megkezdése előtt.

37.4 Csomag kiválasztása és foglalási kérelem leadása (Vendég)

Mire használható: kiválasztott szállodában csomag/szoba(ák) lefoglalása, extra szolgáltatásokkal.

Lépések

- A szálloda oldalon válasszon csomagot (a csomagkártyára kattintva).
- Szükség esetén válasszon további szolgáltatásokat (jelölőnégyzetek).
- Kattintson a „Foglalás megerősítése” gombra.
- A felugró ablakban válassza ki a fizetési módot (banki átutalás vagy bankkártya).
- Adja meg a számlázási adatokat (magánszemély/cég).
- Erősítse meg a foglalási kérelmet.

Mi történik ezután: a foglalás „Függőben” státuszba kerül. A szálloda adminisztrátora elbírálja. Megerősítés után a számla e-mailben érkezik (PDF melléklettel).

37.5 Számla és fizetés folyamata

37.5.1 Bankkártyás fizetés

A számla e-mail tartalmazhat fizetési linket. A link megnyitásával elérhető a „Bankkártyás fizetés” oldal.

- Nyissa meg az e-mailben kapott fizetési linket.
- Ellenőrizze a számla részleteit (sorszámla, összeg, határidő).
- Töltse ki a kártyaadatokat: kártyaszám, lejárat, CVV, kártyatulajdonos neve.
- Kattintson a „Fizetés” gombra.
- Siker esetén a rendszer visszajelzést ad („Fizetés sikeres!”).

37.5.2 Banki átutalás

Banki átutalás esetén a vendég e-mailben kapja meg a számlát (PDF), és a fizetést a számlán szereplő bankszámlaszámra kell teljesíteni. A fizetés beérkezését a szálloda adminisztrátora igazolja vissza.

Fontos: a check-in QR kód e-mail kiküldése a fizetés megerősítése után történik.

37.6 Foglalásaim kezelése (Vendég)

Mire használható: saját foglalások áttekintése, szűrése, vendégek kezelése, számla megtekintése.

Lépések – Foglalások áttekintése

- Nyissa meg a „Foglalásaim” oldalt.
- Használja a szűrőgombokat (Összes / Függőben / Megerősítve / Törölve / Befejezve).
- Válasszon nézetet: „Kártya” vagy „Táblázat”.
- Nyissa meg a foglalás műveleteit a „Műveletek” gombbal (ha elérhető).

Lépések – Vendégek hozzáadása egy foglaláshoz

- Keresse meg a foglalást a listában.
- Kattintson a „+ Vendég hozzáadása” gombra.
- Adja meg a vendég adatait (név, azonosító okmány/ID, születési dátum).

- Mentse a vendéget.

A rendszer jelzi, ha elérte a maximális vendégkapacitást (szobák kapacitása alapján).

37.7 Profil kezelése

Mire használható: név/e-mail módosítás, jelszó csere, 2FA kezelése, fiók törlése.

Lépések – 2FA bekapcsolása

- Nyissa meg a „Profilom” oldalt.
- Kapcsolja be a „Kétfaktoros hitelesítés” opciót.
- Kérjen QR kódot, olvassa be az autentikátor alkalmazással.
- Adja meg a 6 számjegyű kódot, majd erősítse meg.

Szálloda adminoknál a 2FA kötelező lehet az admin felület használatához.

37.8 Szálloda admin: foglalások kezelése (áttekintés)

Mire használható: foglalások elfogadása/elutasítása, számlázási folyamatok kezelése, fizetés megerősítése átutalásnál.

Tipikus lépések

- Lépjen be szálloda admin fiókkal.
- Nyissa meg az Admin → Foglalások oldalt.
- Szűrje a foglalásokat státusz alapján (pl. Függőben / Megerősítve / Fizetésre vár).
- Nyissa meg egy foglalás részleteit/műveleteit, és végezze el a szükséges admin lépést (elfogadás/elutasítás, számla előnézet/jóváhagyás/küldés).
- Átutalás esetén a beérkezett fizetést a rendszerben adminisztrátorként erősítse meg (ezután megy ki a check-in QR e-mail).

37.9 Szálloda admin: számlázási/cégadatok megadása

Mire használható: a számlák kiállításához szükséges adatok rögzítése.

Lépések

- Nyissa meg az Admin → „Cégadatok / Számlázási információk” oldalt.
- Válassza ki a szállodát (ha több szállodája van).
- Töltse ki a kötelező mezőket: Adószám, Bankszámlaszám, Közösségi adószám.
- Kattintson a „Mentés” gombra.

37.10 Super admin: kezelőfunkciók (áttekintés)

A super admin felület célja a rendszer egészének felügyelete. Itt kezelhetők többek között a felhasználók, szállodák, foglalások és egyéb törzsadatok.

Példa: felhasználó hozzáadása (Super admin → Felhasználók):

- Nyissa meg a „Felhasználók kezelése” oldalt.
- Kattintson a „Felhasználó hozzáadása” gombra.
- Adja meg a nevet, e-mail címet, szerepkört, és (új felhasználónál) jelszót.
- Mentse a felhasználót.

38. Űrlapok használata

Általános szabályok:

- A * jelöli a kötelező mezőket (ahol a felület ezt kiírja).
- E-mail mezőbe valós e-mail formátum szükséges (pl. pelda@email.com).
- Jelszó: minimum 8 karakter.
- Dátummezők: érkezés ≤ távozás, és az érkezés nem lehet múltbeli.
- Szám mezők (vendégszám): minimum 1.

38.1 Gyakori mezők és jelentésük

Mező	Mit jelent?	Hol található?
E-mail cím	Bejelentkezéshez és értesítésekhez használt cím	Login / Regisztráció / Profil
Jelszó	Fiók védelme	Login / Regisztráció / Profil / Jelszó visszaállítás
Érkezés / Távozás dátuma	A foglalás időtartama	Keresés / Szálloda oldal
Vendégek száma	Hány főre keres szállást	Keresés / Szálloda oldal
Számlázási adatok	A számlán szereplő adatok (magánszemély/cég)	Foglalás megerősítése (modal)

38.2 Hibakezelés űrlapoknál

Ha egy mező hibás vagy hiányzik:

- A rendszer hibaüzenetet jelenít meg a mező mellett vagy az űrlap tetején.
- Javítsa a megadott adatot a hibaüzenet alapján, majd próbálja újra.

39. Hibaüzenetek és problémák

39.1 Hibás bejelentkezés

Tünet: „Érvénytelen bejelentkezési adatok” vagy hasonló üzenet.

Megoldás:

- Ellenőrizze, hogy helyes-e az e-mail cím.
- Ellenőrizze, hogy a Caps Lock nincs-e bekapcsolva.
- Ha nem emlékszik a jelszóra, használja az „Elfelejtetted a jelszavad?” funkciót.

39.2 E-mail nincs megerősítve

Tünet: a rendszer kéri az e-mail megerősítését.

Megoldás:

- Keresse meg a megerősítő e-mailt a postaládájában (Spam/Promóciók mappát is).
- Kattintson a megerősítő linkre.
- Ha nem érkezett meg, a bejelentkezési oldalon kérje az e-mail újraküldését.

39.3 Hiányzó adatok / hibás formátum

Példák:

- Kötelező mező üresen maradt.
- Dátumok felcserélve vagy múltbeli érkezés.
- Jelszó túl rövid (min. 8 karakter).

Megoldás: javítsa a mező(ke)t, majd mentse/folytassa újra.

39.4 Fizetési oldal hiba

Tünet: a fizetési link megnyitásakor „Nem sikerült betölteni a számla adatait” vagy más hiba.

Megoldás:

- Ellenőrizze, hogy a link teljes és nem sérült (ne hagyjon le karaktereket).
- Próbálja meg másik böngészőben megnyitni.
- Ha továbbra sem működik, kérjen új linket a szállodától (számla újraküldése).

40. Kijelentkezés

A kijelentkezés a fiók biztonsága miatt különösen fontos nyilvános vagy megosztott eszközön.

Lépések

- Keresse meg a navigációban a „Kijelentkezés” opciót (ahol elérhető).
- Kattintson a kijelentkezésre.
- Zárja be a böngészőt, ha megosztott gépet használ.

41. Gyakori kérdések (GYIK)

41.1 Nem kapom meg a megerősítő e-mailt. Mit tegyek?

Ellenőrizze a Spam/Promóciók mappát. A bejelentkezési oldalon kérje az e-mail újraküldését.

41.2 Mikor kapok check-in QR kódot?

A check-in QR kód e-mailt a rendszer a fizetés megerősítése után küldi ki.

41.3 Banki átutalásnál miért nem jön azonnal a QR kód?

Átutalásnál a fizetést a szálloda adminisztrátora igazolja vissza, utána érkezik a QR kód.

41.4 Elfelejtettem a jelszavam. Hogyan tudom visszaállítani?

A bejelentkezés oldalon válassza az „Elfelejtetted a jelszavad?” lehetőséget, és kövesse az e-mailben kapott lépéseket.

Hardver dokumentáció

42. Rendszer rövid áttekintés

A HotelFlow fizikai része felel a szállodai beléptetés megvalósításáért, ezzel kiváltva egy recepciót. Megérkezéskor kiadja a szoba beléptető kártyáját, távozáskor pedig újra eltárolja. A rendszer kezeli a szobákba való belépést is, ezzel egységesítve és egy megoldást nyújtva a szállodáknak.

43. Hardverkomponensek

43.1. Központi egység (kártya tároló szekrény)

A központi egység felel a rendszer teljes működéséért. Ezen keresztül lehet be- és kijelentkezni, valamint a szobáknál leolvasott kártyákat ezen belül hitelesíti a rendszer.

43.1.1. Használt eszközök

Raspberry Pi	1 db
Raspberry Pi Camera	1 db
Arduino Uno	1 db
Relay modul	2 db
Szelenium zár	2 db
LCD kijelző	1 db

43.1.2. Működése

Az eszközben helyett kapott egy Raspberry Pi, ami vezérel mindent. Ezen fut egy Raspberry Pi OS amin 3 Python program fut service ként, így kikapcsolás esetén automatikusan elindul minden. Ezek pedig a következők:

- Be- és kiléptetésért felelős program: adatbázist kezel és szinkronizál a backend adatbázisával
- Kártyák ellenőrzésére szolgáló program: Figyeli az Mosquitto broker topic-jait és az oda érkezett üzenek alapján ellenőrzi a kártyát, majd választ küld rá.
- Fast API végpont: Ennek hívásával tudja a backend elindítani az adatbázis frissítésének folyamatát, változtatás esetén.

A Raspberry saját adatbázisból dolgozik, hogy esetleges hálózati probléma esetén ne szüneteljen a ki és beléptetés. Ez az adatbázis folyamatosan szinkronizálva van a backend adatbázisával. Amikor változás történik, akkor megtörténik a feltöltés. Foglalás esetén a backend megtudja hívni a frissítéshez szükséges végpontot. A helyben futó végpontot Cloudflare Tunnel segítségével éri el a backend, így minimalizálva a biztonsági kockázatot.

A kártyák kiadását végző program, miután megkereste a kártya helyét, üzenetet küld egy arduino-nak ami az ahhoz tartozó szekrényt kinyitja.

43.2 Kártyaolvasó

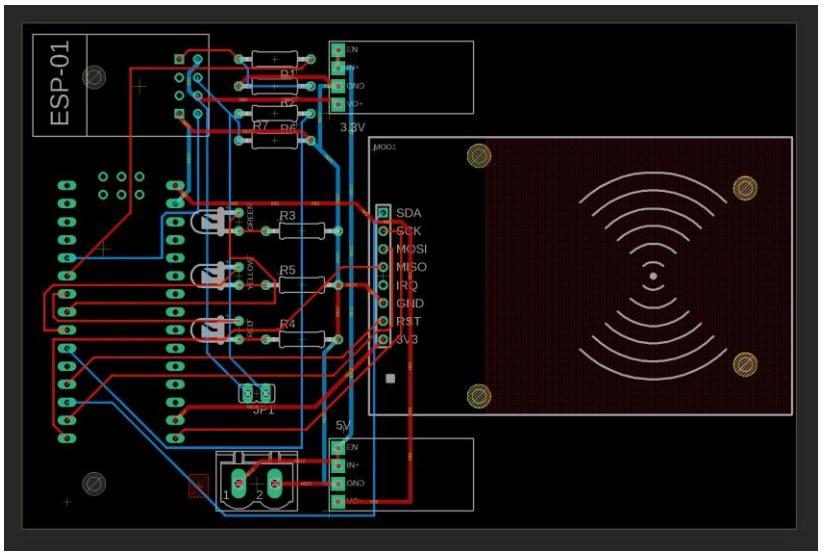
A kártyaolvasó a szoba ajtajánál van elhelyezve. Ez olvassa a beléptető kártyákat, és a hitelesítés után nyitja az ajtót.

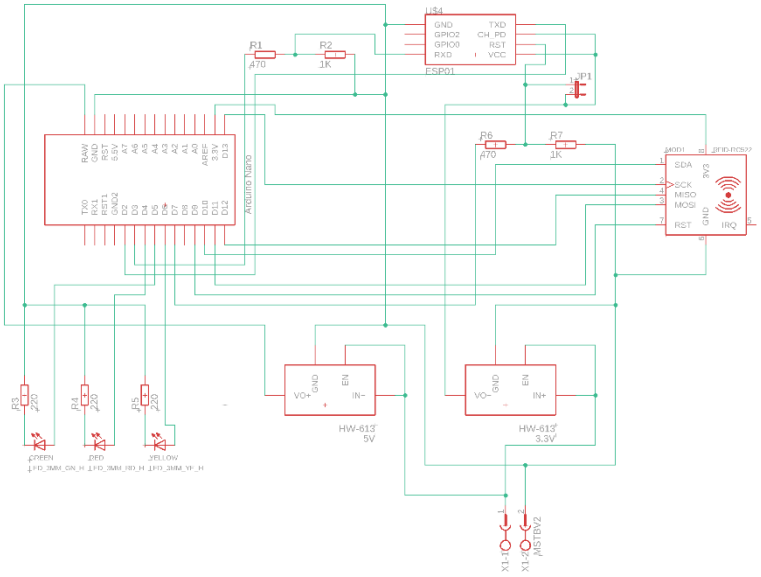
43.2.1. Használt eszközök

Arduino Nano	1 db
ESP-01	1 db
RC522 RFID olvaó	1 db
Step-down modul	2 db
Egyedi PCB	1 db

43.2.2 PCB

A kártyaolvasó eszközökhöz készült egyedi PCB. Ezt a projekt specifikációinak megfelelően terveztük. A gyártást egy ezzel foglalkozó cég végezte el.





43.2.2 Működés

A kiépíthetőséget szemelőt tartva készítettük el a kártyaolvasót. Az eszköznek 12 V-os egyenáramú táplálásra van szüksége, után azt alakítja 5 és 3.3 V-ra, a rajta elhelyezz Step-down modulokkal.

A kártya olvasót, az Arduino nano működteti. A kártya olvasása után az adatokból készít egy json formátumot, amit ellát egy speciális aláírással. Ez az aláírás szolgál az adatok hitelesítése. Ennek a kiszámítását, csak az Arduino és az ellenőrző szoftver ismeri. Miután ezzel ellátta az adatot, azt soros kapcsolaton keresztül továbbítja az ESP-01 felé, ami elküldi a megfelelő Mosquitto topicra. A soros adatküldés során a jelet ellenállások segítségével kellett 3.3 V-ra csökkenteni, ezzel óvva az ESP-t. A visszakapott adatot az ESP továbbítja az Arduino-nak, ami az alapján nyitja az ajtót.

Irodalmi jegyzék

Felhasznált források és eszközök:

Programozási nyelvek és Keretrendszerek

- **Laravel**: A szerveroldali logika, adatkezelés és folyamatok irányításának elsődleges keretrendszere.
- **Vue.js (Vue 3)**: A látványos és könnyen kezelhető felhasználói felület (Frontend) építéséhez használt keretrendszer.
- **PHP**: A Laravel keretrendszer alapjául szolgáló programozási nyelv.
- **JavaScript**: A Vue.js és a frontend interakciók motorja.
- **SQL**: Az adatbázis-lekérdezések és a séma meghatározásának nyelve.
- **C++ és Python**: A projekt adatlapján feltüntetett, kiegészítő fejlesztésekhez vagy modulokhoz használt nyelvek.

Adatbázis-kezelés

- **MySQL**: Az alkalmazás elsődleges relációs adatbázis-kezelő rendszere a foglalások és felhasználói adatok tárolására.
- **PostgreSQL, SQLite**: A dokumentációban említett alternatív, kompatibilis adatbázis-kezelő rendszerek.

Frontend technológiák és Könyvtárak

- **Vite**: A frontend gyors rendereléséért és build folyamataiért felelős eszköz.
- **Axios**: Az API kommunikációt és az adatok lekérését biztosító könyvtár.
- **Vue Router**: A frontend oldali útvonalkezelés és jogosultság alapú navigáció eszköze.
- **Reactive State Management (Pinia/Vue Store)**: A felhasználói munkamenetek és hitelesítési adatok kezelésére használt állapotkezelés.

Fejlesztői eszközök és Verziókezelés

- **GitHub**: A forráskód tárolására és a csapatmunkát segítő verziókezelésre használt platform.
- **Git**: A verziókövetés alapjául szolgáló szoftver.
- **Composer**: A PHP alapú backend függőségek és csomagok kezelésére szolgáló eszköz.
- **NPM / Node.js**: A frontend függőségek telepítéséhez és futtatásához szükséges környezet.

- **Visual Studio Code (VS Code)**: Javasolt kódszerkesztő a webes technológiák fejlesztéséhez.

Tesztelés és Minőségbiztosítás

- **Selenium**: Automatizált E2E (End-to-End) teszteléshez használt eszköz a frontend funkciók ellenőrzésére.
- **API tesztelés (pl. Postman vagy Insomnia)**: A backend végpontok manuális és automatizált ellenőrzésére.
- **Manuális tesztelés**: A felhasználói élmény és a felület közvetlen ellenőrzése.

Biztonság és Hitelesítés

- **Laravel Sanctum**: A token-alapú hitelesítés, a hozzáférések kezelése és a titkosítás biztosítása.
- **Kétfaktoros hitelesítés (2FA)**: A kiemelt jogosultságú fiókok (pl. Super Admin) védelmét szolgáló megoldás.
- **Role-based Access Control (RBAC)**: Szerepkör-alapú hozzáférés-vezérlés (vendég, hotel admin, központi admin).

Hardver és Speciális technológiák

- **RFID technológia**: Érintésmentes kártyás beléptető rendszer megvalósítása a szálláshelyeken.
- **QR-kód alapú Check-in**: Gyors, digitális azonosítás a vendégek érkezésekor.
- **CAD**: A hardver burkolatának megtervezése
- **Eage**: Nyáklap tervezése

Dokumentációk és Szabványok

- **Szoftverspecifikáció**: A rendszer alapvető működését és elvárásait rögzítő alapdokumentum.
- **Adatbázis specifikáció**: A táblák, kapcsolatok és kulcsok technikai leírása.
- **Algoritmus dokumentáció**: A rendszer speciális folyamatainak (pl. foglalás, fizetés) leírása.
- **Felhasználói útmutató**: A vendégek és szállodák részére készült használati segédlet.
- **GDPR**: Az európai adatvédelmi szabályozásnak való megfelelés alapelvei.

Mesterséges Intelligencia (AI) eszközök

- **ChatGPT / OpenAI / Cursor**: A dokumentáció megírásában, kódgenerálásban és hibajavításban nyújtott segítség.
- **GitHub Copilot**: A fejlesztési folyamat során használt AI-asszisztens a kódírás gyorsítására.
- **Claude (Anthropic)**: Összetett logikai összefüggések elemzésére és technikai szövegek szerkesztésére használt AI modell.
- **Gemini**: Grafikai elemek tervezése